

Chapter 6: Markov Chain Monte Carlo Methods

Lectures on Monte Carlo Theory — Lorek & Rolski

Chapter Overview

- 1 6.1 Introduction
- 2 6.2 Markov Chains
- 3 6.3 Constructing MCMC Chains
 - 6.3.1 Metropolis and Metropolis-Hastings
 - 6.3.2 Gibbs Sampler
 - Graph Coloring
 - Hard-core Model
 - Ising Model
- 4 6.4 Perfect Simulation
- 5 6.5 Approximate Counting
- 6 6.6 Computational Biology: Motif Finding
- 7 6.7 Estimating $I = \mathbb{E}f(X)$
- 8 6.8 Stable Simulation Scheme
- 9 6.9 Bibliographical Notes
- 10 6.10 Selected Exercises

6.1 Introduction: The Problem with Direct Sampling I

In the previous chapters the goal was to estimate

$$I = \mathbb{E}Y \quad \text{or} \quad I = \mathbb{E}f(Y),$$

where Y follows a prescribed distribution π (the Greek letter π , pronounced “pie”) on some state space $\mathcal{S}$. The standard Monte Carlo approach generates R independent samples $Y_1, \dots, Y_R \sim \pi$ and uses the sample mean

$$\hat{Y}_R = \frac{1}{R} \sum_{i=1}^R f(Y_i) \approx I.$$

Notation

Here $\hat{Y}_R$ (“Y-hat sub R”) denotes our estimate of I using R samples. The symbol $\sum$ (capital Sigma, meaning “sum”) adds up $f(Y_i)$ for each index i running from 1 to R . The function f is the quantity of interest applied to each sample. The symbol $\approx$ means “approximately equal to”.

This estimator works beautifully when π is easy to sample from directly (e.g. using inverse-CDF or rejection sampling).

6.1 Introduction: The Problem with Direct Sampling II

However, in many real problems the target distribution π is *intractable* for one of two fundamental reasons.

Two Sources of Intractability

- 1 The state space $\mathcal{S}$ is **combinatorially large**: for example, $|\mathcal{S}| = 2^d$ grows exponentially in the problem dimension d (“d”), making enumeration of all states completely hopeless even for moderate d .
- 2 The distribution π involves an **unknown normalising constant**: $\pi_\sigma = e^{\lambda f(\sigma)} / z_\lambda$, where the denominator $z_\lambda = \sum_\sigma e^{\lambda f(\sigma)}$ (a sum over all states σ) cannot be computed because there are too many states to sum over.

In the formula $\pi_\sigma = e^{\lambda f(\sigma)} / z_\lambda$, the symbol σ (sigma, pronounced “sig-ma”) is a specific configuration or state, λ (lambda, pronounced “lam-da”) is a real parameter controlling the shape of the distribution, and z_λ is the *partition function* or *normalising constant* that makes all the probabilities sum to 1.

6.1 Introduction: The Problem with Direct Sampling III

Running example. Consider the set $\mathcal{S}_n$ of all permutations of n (“ n ”) elements. A permutation σ is just an arrangement: for example, with $n = 3$ elements, $\{1, 2, 3\}$, one permutation is $(3, 1, 2)$, meaning element 1 goes to position 3, element 2 goes to position 1, element 3 goes to position 2. There are $n! = 1 \times 2 \times \cdots \times n$ total permutations (e.g. $3! = 6$ for $n = 3$).

In the uniform case, $\pi_\sigma = 1/n!$ for every $\sigma \in \mathcal{S}_n$, and direct sampling is easy (the Fisher-Yates shuffle algorithm). The function of interest counts *fixed points*: $f(\sigma) = \#\{i : \sigma(i) = i\}$, the number of elements that map to themselves.

Now make π non-uniform:

$$\pi_\sigma = \frac{1}{z_\lambda} \exp(\lambda f(\sigma)), \quad z_\lambda = \sum_{\sigma \in \mathcal{S}_n} \exp(\lambda f(\sigma)).$$

6.1 Introduction: The Problem with Direct Sampling IV

Notation for the Non-uniform Case

$\exp(\cdot)$ is the exponential function $e^{(\cdot)}$. The parameter λ (lambda) controls how strongly we weight permutations with many fixed points: when $\lambda > 0$, permutations with more fixed points are assigned higher probability; when $\lambda = 0$ the distribution is uniform. Computing z_λ requires summing over all $n!$ permutations, which is completely infeasible for large n (for $n = 20$, there are $20! \approx 2.4 \times 10^{18}$ permutations).

6.1 Introduction: The Problem with Direct Sampling V

Key Insight: The MCMC Idea

The **Markov Chain Monte Carlo** (MCMC) paradigm resolves the intractability problem by constructing a Markov chain (a sequence of random states) $(X_n)_{n \geq 0}$ on $\mathcal{S}$ whose *stationary distribution* is exactly π . The ergodic theorem (a mathematical guarantee for long-run averages) then ensures

$$\hat{Y}_R = \frac{1}{R} \sum_{j=1}^R f(X_j) \xrightarrow{R \rightarrow \infty} I \quad \text{almost surely.}$$

The symbol $\xrightarrow{R \rightarrow \infty}$ means “converges to as R grows to infinity”. The phrase “almost surely” means “with probability 1” — the convergence happens except on a set of outcomes so rare they have zero probability.

Crucially, the algorithms only need *ratios* $\pi_{s'}/\pi_s = e^{\lambda(f(s')-f(s))}$: the unknown constant z_λ appears in both numerator and denominator and cancels out!

MCMC thus allows us to pursue two goals without ever computing z_λ :

6.1 Introduction: The Problem with Direct Sampling VI

- (A1) Generate approximate (or exact) samples from π .
- (A2) Estimate probabilities $\pi(A) = \sum_{s \in A} \pi_s$ for subsets $A \subseteq \mathcal{S}$.

6.2 Markov Chains: Core Definitions I

A *Markov chain* (named after Russian mathematician Andrei Markov) is a sequence of random variables $(X_n)_{n \geq 0}$ that evolves on a *finite* state space $\mathcal{S} = \{s_1, \dots, s_N\}$ according to the *Markov property*: the future depends on the past only through the current state (the chain has no memory beyond where it currently is).

Definition: Markov Chain and Transition Matrix

The sequence $(X_n)_{n \geq 0}$ is a **Markov chain** with **transition matrix** $\mathbf{P} = (p_{ss'})_{s, s' \in \mathcal{S}}$ if

$$\mathbb{P}(X_{n+1} = s' \mid X_0, X_1, \dots, X_n) = p_{X_n, s'} \quad \text{for all } n \geq 0.$$

Here $p_{ss'}$ (“p sub s, s-prime”) is the probability of moving from state s to state s' in one step, and $\sum_{s'} p_{ss'} = 1$ for every s (the probabilities out of any state sum to 1).

6.2 Markov Chains: Core Definitions II

Notation

The symbol $\mathbb{P}(\cdot \mid \cdot)$ denotes conditional probability: $\mathbb{P}(A \mid B)$ is the probability of event A given that event B has already occurred. The notation $X_0, X_1, \dots, X_n$ is the entire history of the chain up to step n ("n"). The Markov property says that, given the current state X_n , this entire history is irrelevant for predicting X_{n+1} .

6.2 Markov Chains: Core Definitions III

The matrix $\mathbf{P}$ is *time-homogeneous*: the transition probabilities do not change with the step number n . The distribution of the chain at step n is the row vector

$$\boldsymbol{\mu}^n = (\mathbb{P}(X_n = s_1), \dots, \mathbb{P}(X_n = s_N)),$$

and by a simple induction argument $\boldsymbol{\mu}^n = \boldsymbol{\mu}^0 \mathbf{P}^n$, where $\boldsymbol{\mu}^0$ (“mu-naught”, μ is the Greek letter mu pronounced “myoo”) is the *initial distribution*, and $\mathbf{P}^n$ means the transition matrix multiplied by itself n times.

How to Read $\boldsymbol{\mu}^n = \boldsymbol{\mu}^0 \mathbf{P}^n$

This formula says: to find the probability distribution of the chain at step n , start with the initial distribution $\boldsymbol{\mu}^0$ and multiply by the transition matrix n times. Each multiplication by $\mathbf{P}$ advances the chain one step. If you start with all probability on state s_1 (i.e. $\boldsymbol{\mu}^0 = (1, 0, \dots, 0)$), then $\boldsymbol{\mu}^n$ is just the first row of $\mathbf{P}^n$, which gives the probability of being in each state after n steps starting from s_1 .

6.2 Markov Chains: Core Definitions IV

Several structural properties of $\mathbf{P}$ are central to MCMC theory.

6.2 Markov Chains: Core Definitions V

Key Properties of a Transition Matrix

- **Stationary distribution.** A probability vector $\pi = (\pi_s)_{s \in \mathcal{S}}$ (pi, pronounced “pie”) is *stationary* if $\pi \mathbf{P} = \pi$, which means $\pi_{s'} = \sum_s \pi_s p_{ss'}$ for all s' . In words: if the chain starts from π , it stays there forever — $\mu^n = \pi$ for all $n \geq 0$.
- **Irreducibility.** Two states s, s' *communicate* if each can be reached from the other in a finite number of steps: there exist $n_0, n_1 \geq 0$ with $p_{ss'}^{(n_0)} > 0$ and $p_{s's}^{(n_1)} > 0$, where $p_{ss'}^{(n)} = (\mathbf{P}^n)_{ss'}$ (“p sub s,s-prime superscript n”) is the n -step transition probability. The chain is *irreducible* if all states communicate with each other — from any state you can reach any other state.
- **Aperiodicity.** The *period* of state s is $\gcd\{n \geq 1 : p_{ss}^{(n)} > 0\}$ (the greatest common divisor of all step-counts at which the chain can return to s). A period of 1 means the chain can return at any time; a period of 2 means it can only return at even times. The chain is *aperiodic* if every state has period 1.
- **Ergodicity (regularity).** $\mathbf{P}$ is *ergodic* (or *regular*) if there exists some step n_0 such that $\mathbf{P}^{n_0}$ has all strictly positive entries. Equivalently, $\mathbf{P}$ is both irreducible and aperiodic. This is the key property guaranteeing that the chain converges to a unique stationary distribution from any starting point.

The following example illustrates all of these concepts on a simple four-state chain.

6.2 Ergodic Theorem and Convergence I

The following result is the theoretical foundation for all of MCMC. It guarantees that a well-behaved chain will always converge to a unique stationary distribution, regardless of where it starts.

Proposition 6.2.2 (Ergodic Theorem for Markov Chains)

If $\mathbf{P}$ is **regular** (ergodic: irreducible and aperiodic), then there exists a **unique** stationary distribution π with all entries strictly positive ($\pi_s > 0$ for all s), and

$$\mathbf{P}^n \rightarrow \mathbf{\Pi} \quad \text{as } n \rightarrow \infty,$$

where $\mathbf{\Pi}$ (capital Pi, “pie”) is the matrix all of whose rows are equal to π .

Notation

The expression $\mathbf{P}^n \rightarrow \mathbf{\Pi}$ as $n \rightarrow \infty$ means that every entry $p_{ss'}^{(n)}$ of $\mathbf{P}^n$ converges to $\pi_{s'}$ as the number of steps n grows without bound. In other words, after sufficiently many steps, the probability of being in state s' is approximately $\pi_{s'}$, *regardless* of the starting state s .

6.2 Ergodic Theorem and Convergence II

How to Read This Theorem

Think of the chain as a ball bouncing randomly between states. No matter which state the ball starts in, after a long time it will have spent approximately a fraction π_s of its time in state s . The ergodic theorem is what makes MCMC work: it guarantees the long-run average of $f(X_j)$ equals $\mathbb{E}_\pi[f]$.

Corollary 6.2.3

For any initial distribution μ^0 (mu-naught):

$$\mu^0 \mathbf{P}^n \rightarrow \pi \quad \text{as } n \rightarrow \infty.$$

Starting from any distribution, the chain's distribution after n steps converges to π as n grows.

6.2 Ergodic Theorem and Convergence III

To quantify *how fast* the chain converges to π , we use the **total variation distance**, which measures the maximum discrepancy between two probability distributions.

Total Variation Distance and Mixing Time

For two probability vectors μ and ν (“nu”, ν) on $\mathcal{S}$:

$$d_{\text{TV}}(\mu, \nu) = \frac{1}{2} \sum_{s \in \mathcal{S}} |\mu_s - \nu_s| = \max_{A \subseteq \mathcal{S}} |\mu(A) - \nu(A)|.$$

The factor $1/2$ ensures $d_{\text{TV}} \in [0, 1]$: when $d_{\text{TV}} = 0$ the distributions are identical; when $d_{\text{TV}} = 1$ they assign probability zero to entirely different sets of states.

The **mixing time** $\tau(\varepsilon)$ (τ is tau, pronounced “taw”; ε is epsilon, pronounced “ep-si-lon”) is the number of steps needed to get within ε (in total variation distance) of the stationary distribution π :

$$\tau(\varepsilon) = \min \{ n \in \mathbb{N} : d_{\text{TV}}(\mu \mathbf{P}^n, \pi) \leq \varepsilon \}.$$

6.2 Ergodic Theorem and Convergence IV

Notation

$d_{\text{TV}}(\mu, \nu)$ (“d-TV of μ and ν ”) measures how different two probability distributions are. The sum $\sum_{s \in \mathcal{S}} |\mu_s - \nu_s|$ adds the absolute differences in probability for every state. The equivalent formulation $\max_{A \subseteq \mathcal{S}} |\mu(A) - \nu(A)|$ says: find the set A for which the two distributions disagree most about how much probability to assign to A .

How to Read the Mixing Time

The mixing time $\tau(\varepsilon)$ answers the practical question: “How many steps do I need to run the chain before its distribution is close enough to π to be useful?” A small $\tau(\varepsilon)$ means the chain mixes fast (good); a large $\tau(\varepsilon)$ means slow mixing (the chain takes a very long time to forget its starting point, which makes MCMC estimation expensive).

6.2 Ergodic Theorem and Convergence V

The following result shows how to *simulate* a Markov chain in practice using a stream of uniform random numbers.

Proposition 6.2.4 (Update Function Representation)

For any transition matrix $\mathbf{P}$ and initial distribution μ , there exist functions $\varphi : \mathcal{S} \times [0, 1] \rightarrow \mathcal{S}$ (“phi”, the update function) and $\psi : [0, 1] \rightarrow \mathcal{S}$ (“psi”, the initialisation function) such that

$$X_0 = \psi(U_0), \quad X_{n+1} = \varphi(X_n, U_{n+1}),$$

where $U_0, U_1, \dots$ are independent and identically distributed (i.i.d.) uniform random variables on $[0, 1]$.

6.2 Ergodic Theorem and Convergence VI

Notation

φ (phi, “fye”) maps the current state X_n and a fresh random number U_{n+1} to the next state X_{n+1} . ψ (psi, “sigh”) maps the first random number U_0 to the initial state X_0 . $\mathcal{U}[0, 1]$ denotes the uniform distribution on the interval $[0, 1]$. A canonical construction of φ is: $\varphi(s, u) = s_k$ if u falls in the interval $[\sum_{j < k} p_{s, s_j}, \sum_{j \leq k} p_{s, s_j})$, which is the standard inverse-CDF (inverse cumulative distribution function) trick for discrete distributions.

Key Takeaway

Every Markov chain can be driven by a stream of i.i.d. uniform random variables through an update function. This means we can run *multiple chains simultaneously using the same sequence of random numbers*, which is the basis of the coupling idea used in the CFTP perfect simulation algorithm in Section 6.4.

6.2 Detailed Balance and Random Walks on Graphs I

Instead of solving $\pi \mathbf{P} = \pi$ directly (which requires knowing all of $\mathbf{P}$), the *detailed balance condition* (DBC) provides a simpler, pairwise sufficient condition for stationarity.

Proposition 6.2.5 (Detailed Balance Condition)

Suppose there exist strictly positive weights $(w_s)_{s \in \mathcal{S}}$ satisfying

$$w_s p_{ss'} = w_{s'} p_{s's} \quad \text{for all } s, s' \in \mathcal{S}. \quad (\text{DBC})$$

Then $\pi_s = w_s / \sum_{s''} w_{s''}$ is a stationary distribution for $\mathbf{P}$.

Notation

w_s (“w sub s”) is the unnormalised weight assigned to state s . The DBC says: the probability *flow* from s to s' (which is $w_s \cdot p_{ss'}$, the weight of s times the probability of moving from s to s') must exactly equal the flow from s' to s (which is $w_{s'} \cdot p_{s's}$). The stationary probability $\pi_s = w_s / \sum_{s''} w_{s''}$ is obtained by normalising the weights so they sum to 1.

6.2 Detailed Balance and Random Walks on Graphs II

How to Read the DBC

Imagine water flowing through a network of pipes, where each state is a reservoir and each transition is a pipe. The DBC says that at stationarity, the flow in each pipe is the same in both directions: what flows from s to s' exactly balances what flows from s' to s . This “balance” condition ensures the water level (probability) at each reservoir stays constant over time.

Proof sketch. Setting $w_s = \pi_s$ and summing the DBC over all s :

$\sum_s \pi_s p_{ss'} = \sum_s \pi_{s'} p_{s's} = \pi_{s'} \sum_s p_{s's} = \pi_{s'}$. This is exactly the stationary condition $\pi \mathbf{P} = \pi$. $\square$

The DBC is equivalent to **time-reversibility**: if $X_0 \sim \pi$ then the sequence $(X_0, X_1, \dots, X_n)$ has the same joint distribution as its time-reverse $(X_n, X_{n-1}, \dots, X_0)$. A chain satisfying DBC is called *reversible* because running it backward in time gives the same statistics as running it forward.

Corollary 6.2.6

If $\mathbf{P}$ is symmetric ($p_{ss'} = p_{s's}$ for all s, s'), then the stationary distribution is *uniform*: $\pi_s = 1/|\mathcal{S}|$ for all s .

6.2 Detailed Balance and Random Walks on Graphs III

Proof. Take $w_s = 1$ for all s . Then $w_s p_{ss'} = p_{ss'} = p_{s's} = w_{s'} p_{s's}$ by symmetry. Normalising gives $\pi_s = 1/|\mathcal{S}|$. $\square$

6.2 Detailed Balance and Random Walks on Graphs IV

Example 6.2.7: Simple Random Walk on a Graph $G = (V, E)$

A graph G consists of a set of vertices V (points) and edges E (connections between pairs of points). The *degree* $\deg(v)$ of vertex v is the number of edges attached to v . The simple random walk moves from the current vertex v to any of its neighbours (directly connected vertices) with equal probability:

$$q_{vv'} = \begin{cases} \frac{1}{\deg(v)}, & \text{if } \{v, v'\} \in E \text{ (they are connected),} \\ 0, & \text{otherwise.} \end{cases}$$

Setting $w_v = \deg(v)$ and checking the DBC: $w_v q_{vv'} = \deg(v) \cdot \frac{1}{\deg(v)} = 1 = \deg(v') \cdot \frac{1}{\deg(v')} = w_{v'} q_{v'v}$. Thus the DBC holds and the stationary distribution is

$$\pi_v = \frac{\deg(v)}{\sum_{v'} \deg(v')} = \frac{\deg(v)}{2|E|}.$$

The stationary distribution assigns probability proportional to the degree of each vertex. On a regular graph (where every vertex has the same degree Δ), π is uniform.

Example 6.2.9: Adding Self-loops for Aperiodicity

6.3 The MCMC Construction Problem I

We now address the central challenge of MCMC: given a target distribution π on a large or complex state space $\mathcal{S}$ where direct sampling is impossible, how do we construct a Markov chain that (a) has π as its stationary distribution, and (b) is easy to simulate step by step?

The key observation is that to verify the DBC or to compute acceptance ratios in the algorithms below, we only ever need to evaluate *ratios* $\pi_{s'}/\pi_s$. If $\pi_s = c \cdot \tilde{\pi}_s$ for some known unnormalised weights $\tilde{\pi}_s$ but unknown normalising constant c , the constant cancels:

$$\frac{\pi_{s'}}{\pi_s} = \frac{c \cdot \tilde{\pi}_{s'}}{c \cdot \tilde{\pi}_s} = \frac{\tilde{\pi}_{s'}}{\tilde{\pi}_s}.$$

How to Read This Cancellation

The fact that the unknown constant c appears in both numerator and denominator and cancels is the single most important practical observation in MCMC. It means we can work with π even when we only know it up to a multiplicative constant — which is exactly the situation in Bayesian statistics, statistical physics, and combinatorics.

6.3 The MCMC Construction Problem II

The two canonical MCMC algorithms are:

- **Metropolis-Hastings** (Section 6.3.1): propose a move to a new state from an arbitrary “proposal” distribution, then accept or reject that proposed move based on a ratio test. The ratio test is designed so that the chain has π as its stationary distribution, regardless of the proposal chosen.
- **Gibbs sampler** (Section 6.3.2): applicable when the state space has a product structure $\mathcal{S} = C^V$ (every configuration assigns a value from alphabet C to each vertex in set V). The Gibbs sampler updates one coordinate at a time, sampling from the exact conditional distribution of that coordinate given all others. No acceptance/rejection is needed: every proposal is accepted.

Both algorithms produce a Markov chain satisfying the DBC with weights $w_s = \pi_s$, and therefore having stationary distribution exactly π .

6.3.1 The Metropolis Algorithm I

Setting. We are given a target distribution π on $\mathcal{S}$ and a *symmetric*, regular transition matrix $\mathbf{Q} = (q_{ss'})$ called the *candidate-generating matrix* or *proposal matrix*. The word “symmetric” here means $q_{ss'} = q_{s's}$ for all s, s' : the probability of proposing a move from s to s' is the same as from s' to s . From the current state s , we propose moving to a new state s' drawn from the distribution $\mathbf{q}_s$ (the row of $\mathbf{Q}$ corresponding to state s). The proposal is then either accepted (and we move to s') or rejected (and we stay at s), with an acceptance probability that ensures π is the stationary distribution.

The Metropolis Transition Matrix

The resulting transition matrix $\mathbf{P}$ has entries:

$$p_{ss'} = \begin{cases} q_{ss'} \min\left(1, \frac{\pi_{s'}}{\pi_s}\right), & s \neq s', \\ 1 - \sum_{k \neq s} q_{sk} \min\left(1, \frac{\pi_k}{\pi_s}\right), & s = s'. \end{cases}$$

6.3.1 The Metropolis Algorithm II

Notation

For $s \neq s'$: $p_{ss'}$ is the probability of moving from s to s' , which equals the probability $q_{ss'}$ of proposing s' times the probability $\min(1, \pi_{s'}/\pi_s)$ of accepting. For $s = s'$: p_{ss} is the probability of staying put, which is 1 minus the total probability of moving anywhere. The function $\min(1, x)$ equals 1 when $x \geq 1$ and equals x when $x < 1$.

How to Read the Acceptance Probability $\min(1, \pi_{s'}/\pi_s)$

If the proposed state s' has higher probability than the current state s (i.e. $\pi_{s'} > \pi_s$), the ratio $\pi_{s'}/\pi_s > 1$ and the move is always accepted (probability 1). If the proposed state s' has lower probability, the move is accepted with probability $\pi_{s'}/\pi_s < 1$. This asymmetry means the chain spends more time in high-probability states, which is exactly what we want: we are trying to sample from π , so high- π states should be visited more often.

6.3.1 The Metropolis Algorithm III

Algorithm 42 (Metropolis, one step)

Given: Current state $Y_k = s$; symmetric proposal matrix $\mathbf{Q}$; target π .

- ① Sample a proposed new state X from the proposal distribution $\mathbf{q}_s = (q_{s,s_1}, \dots, q_{s,s_N})$.
- ② Compute the acceptance probability α (alpha) $= \min(1, \pi_X/\pi_s)$.
- ③ Draw an independent uniform random variable $U \sim \mathcal{U}[0, 1]$.
- ④ **Accept** the proposal (set $Y_{k+1} = X$) if $U \leq \alpha$; otherwise **reject** (set $Y_{k+1} = Y_k = s$).

Why does this work? We verify the detailed balance condition (DBC) with weights $w_s = \pi_s$. For any two distinct states $s \neq s'$ with $q_{ss'} > 0$:

$$\pi_s p_{ss'} = \pi_s q_{ss'} \min(1, \pi_{s'}/\pi_s) = q_{ss'} \min(\pi_s, \pi_{s'}).$$

By symmetry of $\mathbf{Q}$ ($q_{ss'} = q_{s's}$) and symmetry of the min function:

$$\pi_{s'} p_{s's} = q_{s's} \min(\pi_{s'}, \pi_s) = q_{ss'} \min(\pi_s, \pi_{s'}) = \pi_s p_{ss'}. \checkmark$$

6.3.1 The Metropolis Algorithm IV

Proposition 6.3.1 (Metropolis)

If $\mathbf{Q}$ is symmetric and regular, then the Metropolis transition matrix $\mathbf{P}$ is regular with stationary distribution π .

6.3.1 The Metropolis Algorithm V

Worked Example: Two-State Metropolis

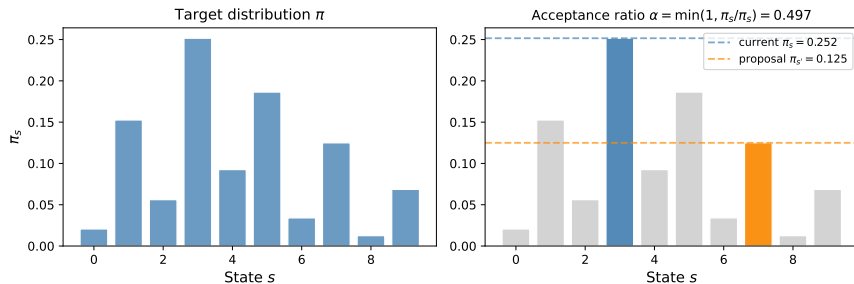
Let $\mathcal{S} = \{0, 1\}$, $\pi_0 = 0.3$ (30% probability on state 0), $\pi_1 = 0.7$ (70% on state 1), and $\mathbf{Q} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (the only proposal is to switch states).

From state 0: the proposed state is 1. Acceptance probability
 $= \min(1, \pi_1/\pi_0) = \min(1, 0.7/0.3) = \min(1, 2.33) = 1$. Since $\pi_1 > \pi_0$ (state 1 is more probable), we always accept a move from the less-probable state 0 to the more-probable state 1. So $p_{01} = 1 \cdot 1 = 1$.

From state 1: the proposed state is 0. Acceptance probability
 $= \min(1, \pi_0/\pi_1) = \min(1, 0.3/0.7) = 3/7 \approx 0.43$. So $p_{10} = 1 \cdot 3/7 = 3/7$ and $p_{11} = 1 - 3/7 = 4/7$ (stay with probability 4/7).

DBC check: $\pi_0 \cdot p_{01} = 0.3 \times 1 = 0.3$ and $\pi_1 \cdot p_{10} = 0.7 \times 3/7 = 0.3$. They are equal, confirming DBC. ✓

6.3.1 The Metropolis Algorithm VI



6.3.1 Metropolis-Hastings: Asymmetric Proposals I

The Metropolis algorithm requires a symmetric proposal $\mathbf{Q}$ (so that $q_{ss'} = q_{s's}$). This restriction is often inconvenient: natural proposals on graphs, for example, may visit high-degree vertices more often than low-degree ones. The **Metropolis-Hastings** (MH) algorithm generalises Metropolis by allowing *any* regular proposal matrix and introducing a correction factor to compensate for the proposal's asymmetry.

Algorithm 43 (Metropolis-Hastings, one step)

Given: Current state $Y_k = s$; regular (possibly asymmetric) proposal matrix $\mathbf{Q}$; target π .

- 1 Sample proposed state X from $\mathbf{q}_s$ (the row of $\mathbf{Q}$ for state s).
- 2 Compute the **Hastings acceptance ratio**:

$$\alpha_{sX} = \min\left(1, \frac{\pi_X q_{Xs}}{\pi_s q_{sX}}\right).$$

- 3 Draw $U \sim \mathcal{U}[0, 1]$ independently. If $U \leq \alpha_{sX}$: set $Y_{k+1} = X$; else set $Y_{k+1} = Y_k = s$.

6.3.1 Metropolis-Hastings: Asymmetric Proposals II

Notation for the Hastings Acceptance Ratio

In the ratio $\frac{\pi_X q_{Xs}}{\pi_s q_{sX}}$:

- π_X is the target probability of the proposed state X .
- π_s is the target probability of the current state s .
- q_{sX} is the proposal probability of moving from s to X (how likely is this proposal?).
- q_{Xs} is the proposal probability of moving from X back to s (how likely would the reverse proposal be?).

The ratio q_{Xs}/q_{sX} is the correction factor for the proposal's asymmetry. If the proposal $\mathbf{Q}$ is symmetric ($q_{sX} = q_{Xs}$), this factor is 1 and we recover the original Metropolis acceptance probability $\min(1, \pi_X/\pi_s)$.

6.3.1 Metropolis-Hastings: Asymmetric Proposals III

How to Read the Hastings Ratio

The factor $\pi_X q_{Xs} / (\pi_s q_{sX})$ corrects for the asymmetry in the proposal: if $\mathbf{Q}$ tends to over-propose moves from s to X (i.e. $q_{sX} \gg q_{Xs}$, so we propose this direction much more than the reverse), the correction factor downweights the acceptance probability accordingly. This correction is essential: without it, the chain would spend too much time in states that are over-proposed, rather than spending time proportional to π .

6.3.1 Metropolis-Hastings: Asymmetric Proposals IV

Proposition 6.3.2 (Metropolis-Hastings)

Let $\mathbf{Q}$ be regular with $q_{ss'} > 0 \Leftrightarrow q_{s's} > 0$ (transitions are possible in both directions or neither). Then the MH transition matrix

$$p_{ss'} = q_{ss'} \alpha_{ss'} \quad (s \neq s'), \quad \alpha_{ss'} = \min\left(1, \frac{\pi_{s'} q_{s's}}{\pi_s q_{ss'}}\right),$$

is regular with stationary distribution π .

Proof (DBC). For $s \neq s'$:

$$\pi_s p_{ss'} = \pi_s q_{ss'} \min\left(1, \frac{\pi_{s'} q_{s's}}{\pi_s q_{ss'}}\right) = \min(\pi_s q_{ss'}, \pi_{s'} q_{s's}).$$

By symmetry of min:

$$\pi_{s'} p_{s's} = \pi_{s'} q_{s's} \min\left(1, \frac{\pi_s q_{ss'}}{\pi_{s'} q_{s's}}\right) = \min(\pi_{s'} q_{s's}, \pi_s q_{ss'}) = \pi_s p_{ss'}. \quad \square$$

6.3.1 Metropolis-Hastings: Asymmetric Proposals V

Special case: Random Walk MH on a graph. On a connected graph $G = (V, E)$, take $q_{vv'} = 1/K$ for neighbours $v' \in \mathcal{N}(v)$ (where $\mathcal{N}(v)$ is the set of neighbours of v), and $q_{vv} = 1 - \deg(v)/K$ for a self-loop, with $K \geq \Delta = \max \deg$ (the maximum degree over all vertices). To sample from a target $\pi_v = b_v/B$ with $B = \sum_v b_v$ unknown, the acceptance ratio becomes:

$$\alpha_{vv'} = \min\left(1, \frac{(\pi_{v'}/K)}{(\pi_v/K)}\right) = \min\left(1, \frac{\pi_{v'}}{\pi_v}\right) = \min\left(1, \frac{b_{v'}}{b_v}\right).$$

The unknown normalising constant B cancels completely, and we never need to compute it.

Key Insight: Normalising Constant Always Cancels

In any MH algorithm, the target π appears only through ratios $\pi_{s'}/\pi_s$. If $\pi_s = \tilde{\pi}_s/Z$ for some known $\tilde{\pi}_s$ and unknown Z , then $\pi_{s'}/\pi_s = \tilde{\pi}_{s'}/\tilde{\pi}_s$ — the unknown Z cancels. This is the single most important practical advantage of MCMC over direct sampling.

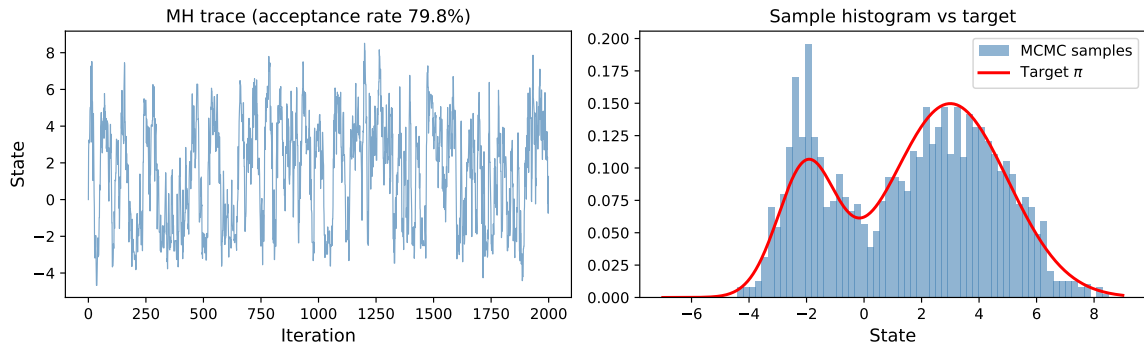
6.3.1 Metropolis-Hastings: Python Implementation

```
import numpy as np

def metropolis_hastings(log_pi, proposal, x0, n_steps):
    """
    Generic Metropolis-Hastings sampler.
    log_pi : callable, log of (unnormalized) target density
    proposal: callable, proposal(x) -> (x_new, log_q_ratio)
              where log_q_ratio = log q(x_new->x) - log q(x->x_new)
    x0      : initial state
    n_steps : number of steps
    """
    samples = [x0]
    x = x0
    n_accept = 0
    for _ in range(n_steps):
        x_new, log_q_ratio = proposal(x)
        # MH acceptance ratio (in log space for numerical stability)
        log_alpha = (log_pi(x_new) - log_pi(x)) + log_q_ratio
        if np.log(np.random.rand()) < log_alpha:
            x = x_new
            n_accept += 1
        samples.append(x)
    print(f"Acceptance rate: {n_accept/n_steps:.1%}")
    return np.array(samples)

# Example: mixture of two Gaussians (unknown normalizing constant)
def log_pi_mix(x):
```

6.3.1 MH Trace: Convergence in Practice I



Three distinct phases are visible in a typical MCMC run, and understanding them is essential for correct use of the output.

- 1 **Burn-in:** In the initial phase, the chain has not yet reached its stationary distribution. The samples are biased toward the starting state x_0 and should be *discarded* rather than included in the average. How long the burn-in lasts depends on the mixing time $\tau(\varepsilon)$.

6.3.1 MH Trace: Convergence in Practice II

- ② **Mixing:** Once the chain has forgotten its starting state, it begins to traverse the full support of π , visiting high-probability regions frequently and low-probability regions rarely (in proportion to π). The running average $\hat{Y}_r$ begins to stabilise.
- ③ **Estimation:** We average $f(X_j)$ over the post-burn-in samples. Because consecutive samples are correlated (the chain does not jump randomly — it moves by small steps), the effective sample size is smaller than the nominal sample size R .

The choice of proposal step size σ (sigma) is crucial for performance: too small a σ leads to slow exploration (high acceptance rate, but the chain barely moves); too large a σ leads to many rejections (low acceptance rate, chain stays put). For a random-walk proposal in $\mathbb{R}^d$ (“d-dimensional real space”), theory shows the optimal acceptance rate is approximately 23%.

Key Takeaway

The burn-in period is discarded; the remaining samples form a (correlated but valid) sequence from $\approx \pi$. A practical convergence diagnostic: monitor $\hat{Y}_R$ as a running average; once it stabilises and stops trending, the chain has mixed.

6.3.2 The Gibbs Sampler: Setup and Motivation I

The Gibbs sampler is an MCMC algorithm specifically designed for distributions on product spaces. It applies when the state space has the form $\mathcal{S} = C^V$, where V is a finite set of “sites” or “vertices” (coordinates), and $C = \{1, \dots, r\}$ is a finite “alphabet” of values each site can take. A *configuration* $\xi \in \mathcal{S}$ (xi, pronounced “ksee”) is a function $\xi : V \rightarrow C$ that assigns a value $\xi(v) \in C$ to each site $v \in V$. Many important models have this product-space structure: spin systems in statistical physics (where each vertex is an atom with a spin), Markov random fields in machine learning (image segmentation, etc.), and combinatorial models such as graph colourings and independent sets. The key idea of the Gibbs sampler is simple: instead of proposing a new *entire* configuration, update one *coordinate* (site) at a time, sampling it from its exact conditional distribution given all other coordinates. Because this conditional distribution comes from π exactly, no acceptance/rejection step is needed.

6.3.2 The Gibbs Sampler: Setup and Motivation II

Full Conditional Distribution

Given a configuration $\xi \in \mathcal{S}$ and a site $v \in V$, we write ξ_{-v} (“xi minus v”) for the restriction of ξ to all sites except v (i.e., the values of all other coordinates). The **full conditional distribution** of coordinate v given all others is

$$p_{\xi,v}(c) = \mathbb{P}(Z(v) = c \mid Z_{-v} = \xi_{-v}), \quad c \in C,$$

where $Z \sim \pi$ is a random configuration drawn from π . In words: this is the probability that the value at site v equals c , given that all other sites take the values specified by ξ .

6.3.2 The Gibbs Sampler: Setup and Motivation III

Algorithm 44 (Gibbs Sampler, one step)

Given: Current configuration $X_k = \xi$; target distribution π .

- 1 Choose a site $v \in V$ uniformly at random (each site is equally likely to be updated).
- 2 For each possible value $c \in C$, compute the weight of the modified configuration $\xi(v \rightarrow c)$ (meaning: ξ with the value at site v changed to c , all other values unchanged). The weight is proportional to $\pi_{\xi(v \rightarrow c)}$.
- 3 Normalise: $p_{\xi, v}(c) = \pi_{\xi(v \rightarrow c)} / \sum_{c' \in C} \pi_{\xi(v \rightarrow c')}$. This is the full conditional.
- 4 Sample $c^* \sim p_{\xi, v}(\cdot)$ (draw a random value from the full conditional).
- 5 Set $X_{k+1}(v) = c^*$ (update site v) and $X_{k+1}(w) = \xi(w)$ for all $w \neq v$ (leave all other sites unchanged).

6.3.2 The Gibbs Sampler: Setup and Motivation IV

Notation

$\xi(v \rightarrow c)$ (“xi with v replaced by c ”) is the configuration identical to ξ at every site except v , where it takes value c instead. The normalisation denominator $\sum_{c' \in C} \pi_{\xi(v \rightarrow c')}$ sums the weights of all possible values at site v (with all other sites fixed), and equals the marginal probability of the pattern ξ_{-v} at all sites except v . Crucially, any unknown normalising constant in π cancels in this ratio, so we only need to evaluate $\pi_{\xi(v \rightarrow c)}$ up to a constant.

6.3.2 The Gibbs Sampler: Setup and Motivation V

Proposition 6.3.5 (Gibbs Sampler Correctness)

If the Gibbs sampler chain is ergodic (which must be verified for each specific model), its unique stationary distribution is π .

Proof (DBC). We verify DBC for any two configurations ξ, ξ' that differ at exactly one site v (so $\xi'(v) \neq \xi(v)$ but $\xi'(w) = \xi(w)$ for $w \neq v$):

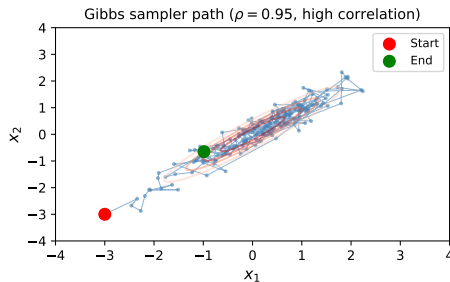
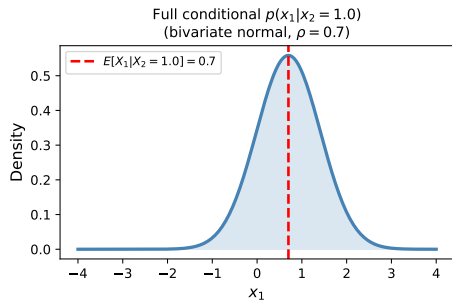
$$\pi_{\xi} \cdot p_{\xi\xi'} = \pi_{\xi} \cdot \frac{1}{|V|} \cdot p_{\xi,v}(\xi'(v)) = \pi_{\xi} \cdot \frac{1}{|V|} \cdot \frac{\pi_{\xi'}}{\sum_c \pi_{\xi(v \rightarrow c)}} = \frac{\pi_{\xi} \pi_{\xi'}}{|V| \sum_c \pi_{\xi(v \rightarrow c)}}.$$

This expression is symmetric in ξ and ξ' (swapping them gives the same formula), so DBC holds. $\square$

6.3.2 The Gibbs Sampler: Setup and Motivation VI

Note on Ergodicity

The Gibbs sampler can fail to be ergodic (e.g., be reducible or periodic) for specific models. For instance, if the alphabet $\mathcal{C}$ has only one valid value at every site (all states are determined), the chain never moves. Regularity (irreducibility and aperiodicity) must be checked for each specific model separately.



6.3.2.1 Graph Coloring I

Problem. Given a graph $G = (V, E)$ and a colour set $C = \{1, \dots, r\}$, a *proper r -colouring* is an assignment $\xi : V \rightarrow C$ of colours to vertices such that no two adjacent (directly connected) vertices share the same colour: $\xi(v) \neq \xi(v')$ for all $\{v, v'\} \in E$. The number of proper r -colourings of G is denoted $z_G = |\Lambda|$, where Λ is the set of all valid colourings.

Target Distribution: Uniform over Proper Colourings

$$\pi_\xi = \frac{1}{z_G} \mathbf{1}(\xi \in \Lambda).$$

Here $\mathbf{1}(\xi \in \Lambda)$ (“indicator of ξ in Λ ”) is 1 if ξ is a valid colouring and 0 otherwise. So every valid colouring has equal probability $1/z_G$, and invalid colourings have zero probability. Computing z_G exactly is #P-complete in general (believed to be computationally harder than NP-complete), but sampling from π is tractable via Gibbs sampling when r is large enough.

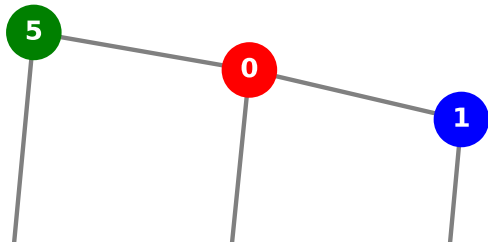
6.3.2.1 Graph Coloring II

Gibbs sampler for graph colouring (Algorithm 45). Given the current colouring ξ , pick one vertex $v \in V$ uniformly at random. Let $\mathcal{C}_{\xi,v} = \{\xi(w) : w \in \mathcal{N}(v)\}$ be the set of colours already used by v 's neighbours. These are the *forbidden* colours for v . The full conditional distribution is then uniform over all *available* (non-forbidden) colours:

$$p_{\xi,v}(c) = \begin{cases} 0, & c \in \mathcal{C}_{\xi,v} \text{ (colour already used by a neighbour),} \\ \frac{1}{r - |\mathcal{C}_{\xi,v}|}, & c \in C \setminus \mathcal{C}_{\xi,v} \text{ (colour is available).} \end{cases}$$

We simply pick c^* uniformly from the available colours $C \setminus \mathcal{C}_{\xi,v}$ and set $\xi(v) \leftarrow c^*$.

Proper 3-coloring of a graph



The Gibbs sampler for graph colouring requires $r \geq \Delta + 1$ colours to ensure there is always at least one available colour at every vertex, where $\Delta = \max_{v \in V} \deg(v)$ is the maximum degree. If $r < \Delta + 1$, a vertex with Δ neighbours might have all r colours forbidden.

Metropolis alternative (Algorithm 46):

choose $v \in V$ and $c \in C$ uniformly at random;
accept the new colouring $\xi(v \leftarrow c)$ if it is still

6.3.2.1 Graph Coloring III

Rapid Mixing (Lemma 6.5.15)

If $r \geq 2\Delta + 1$ (at least twice the maximum degree plus one), the Metropolis chain on colourings is **rapidly mixing** with mixing time

$$\tau(\kappa) \leq \frac{r}{r - 2\Delta} M \ln\left(\frac{M}{\kappa}\right), \quad M = |V|.$$

Here κ (kappa, “kap-a”) is the desired accuracy in total variation distance, $\ln$ is the natural logarithm, and $M = |V|$ is the number of vertices.

Notation

A chain is **rapidly mixing** if its mixing time $\tau(\varepsilon)$ grows polynomially (not exponentially) in the problem size M and $\ln(1/\varepsilon)$. The bound above is polynomial in M and $\ln(M/\kappa)$, so the chain reaches stationarity efficiently even for large graphs.

6.3.2.1 Graph Coloring IV

Remark on Ergodicity

If $r = \Delta$ exactly, the chain may be periodic or reducible. For example, on the complete graph K_3 (three vertices, each pair connected) with $r = 3$ colours, every valid 3-colouring assigns a distinct colour to each vertex, and no vertex can change its colour without creating a conflict. The chain is then “trapped” and is not ergodic. Always verify $r > \Delta$ to ensure ergodicity of the Gibbs sampler.

6.3.2.2 Hard-core Model I

Problem. Given a graph $G = (V, E)$ and a parameter $\lambda > 0$ (lambda, “lam-da”), each vertex is either *occupied* ($\xi(v) = 1$, representing a particle or an agent) or *empty* ($\xi(v) = 0$). A configuration $\xi \in \{0, 1\}^V$ is *feasible* (an *independent set*) if no two adjacent vertices are both occupied: $\xi(v) = \xi(w) = 1 \Rightarrow \{v, w\} \notin E$. This models the “hard-core” constraint that particles cannot be adjacent (like molecules that physically cannot overlap).

Generalised Hard-core Distribution

$$\pi_\xi = \frac{\lambda^{m(\xi)}}{z_{G,\lambda}} \mathbf{1}(\xi \in \Lambda), \quad z_{G,\lambda} = \sum_{\xi \in \Lambda} \lambda^{m(\xi)},$$

where $m(\xi) = \sum_{v \in V} \xi(v)$ is the number of occupied vertices in configuration ξ , and Λ is the set of all independent sets of G . The parameter λ is called the *activity*: when $\lambda = 1$ this is the uniform distribution over all independent sets; when $\lambda > 1$, configurations with more particles have higher probability (the “gas” is denser); when $\lambda < 1$, fewer particles are favoured.

6.3.2.2 Hard-core Model II

Notation

$\lambda^{m(\xi)}$ (“lambda to the power $m(\xi)$ ”) is a weight that increases with the number of particles when $\lambda > 1$. $z_{G,\lambda}$ is the partition function (normalising constant), which is intractable to compute for large graphs. $\mathbf{1}(\xi \in \Lambda)$ enforces the hard-core constraint: only valid independent sets have positive probability.

6.3.2.2 Hard-core Model III

Full conditional for the hard-core model. Given configuration ξ and a vertex v , the conditional probability of the value at v (given all other vertices) depends on whether v 's neighbours are occupied:

- If *any* neighbour $w \in \mathcal{N}_G(v)$ is occupied ($\xi(w) = 1$): the hard-core constraint forbids v from being occupied, so $p_{\xi,v}(1) = 0$ and $p_{\xi,v}(0) = 1$ (vertex v must be empty).
- If *all* neighbours of v are empty ($\xi(w) = 0$ for all $w \in \mathcal{N}_G(v)$): there is no constraint, and the conditional probability is proportional to the weight λ^m , giving:

$$p_{\xi,v}(1) = \frac{\lambda}{\lambda + 1}, \quad p_{\xi,v}(0) = \frac{1}{\lambda + 1}.$$

To derive the conditional: the weight of occupying v (with all neighbours empty) is $\lambda^{m(\xi)+1} \cdot 1 = \lambda$ times more than the weight of leaving it empty (weight $\lambda^{m(\xi)} \cdot 1$). Normalising: $p_{\xi,v}(1) = \lambda/(\lambda + 1)$.

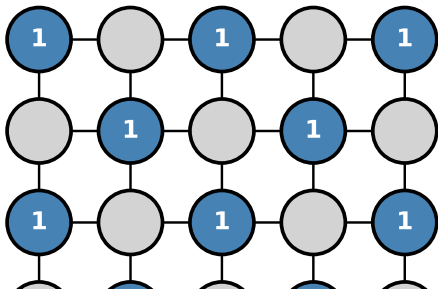
6.3.2.2 Hard-core Model IV

Algorithm 47 (Gibbs Sampler for Hard-core Model)

Given: Graph $G = (V, E)$, parameter $\lambda > 0$.

- 1 Choose $v \in V$ uniformly at random. Flip a biased coin that lands heads with probability $\lambda/(\lambda + 1)$.
- 2 If any neighbour of v is occupied: forcibly set $\xi(v) = 0$ (regardless of the coin).
- 3 If all neighbours of v are empty: set $\xi(v) = 1$ on heads, $\xi(v) = 0$ on tails.

Hard-core model: independent set on grid



The figure shows typical configurations of the hard-core model on a grid graph for different values of λ (lambda). For small λ (low activity), most vertices are empty. For large λ (high activity), configurations concentrate on maximum independent sets, which on a grid look like “checkerboard” patterns where every other vertex is occupied.

6.3.2.4 Ising Model I

The Ising model (named after physicist Ernst Ising, who solved the 1D case in 1925) is one of the most studied models in statistical physics and serves as a canonical example for Gibbs sampling. It models magnetic materials: each atom has a “spin” that can point up or down, and neighbouring atoms tend to align.

Setup. Consider a graph $G = (V, E)$ (typically a 2D grid) where each vertex $v \in V$ carries a *spin* $\xi(v) \in \{-1, +1\}$. A spin of $+1$ represents “up” and -1 represents “down”.

6.3.2.4 Ising Model II

Ising Distribution

The *energy* of a configuration $\xi \in \{-1, +1\}^V$ is

$$H(\xi) = - \sum_{\{v, v'\} \in E} \xi(v) \xi(v').$$

The sum is over all edges $\{v, v'\} \in E$ (pairs of adjacent vertices). When two adjacent vertices have the same spin ($\xi(v) = \xi(v') = +1$ or both -1), they contribute -1 to H (lowering the energy). When they have opposite spins, they contribute $+1$ (raising the energy). So low-energy configurations are those where neighbours align.

The Ising distribution is the Gibbs measure with inverse temperature β (beta, “bay-ta”):

$$\pi_\xi = \frac{1}{Z_\beta} \exp\left(\beta \sum_{\{v, v'\} \in E} \xi(v) \xi(v')\right) = \frac{e^{-\beta H(\xi)}}{Z_\beta}.$$

When $\beta > 0$ (ferromagnet), aligned configurations have higher probability. When $\beta < 0$ (anti-ferromagnet), alternating configurations are favoured.

6.3.2.4 Ising Model: Python Implementation

```
import numpy as np
import matplotlib.pyplot as plt

def ising_gibbs(n, beta, n_steps=100000, seed=0):
    """
    Gibbs sampler for Ising model on n x n periodic grid.
    n      : grid size (n x n vertices)
    beta   : inverse temperature (>0 ferromagnet, <0 antiferromagnet)
    """
    np.random.seed(seed)
    config = np.random.choice([-1, 1], size=(n, n))

    for step in range(n_steps):
        i, j = np.random.randint(0, n, 2)
        # Sum of four neighboring spins (periodic boundary conditions)
        nbr_sum = (config[(i-1)%n, j] + config[(i+1)%n, j]
                  + config[i, (j-1)%n] + config[i, (j+1)%n])
        # Full conditional: P(spin = +1 | neighbors)
        # alpha = 1 / (1 + exp(-2*beta*nbr_sum))
        h = beta * nbr_sum
        prob_plus = np.exp(h) / (np.exp(h) + np.exp(-h))
        config[i, j] = 1 if np.random.rand() < prob_plus else -1

    return config

# Run near the critical temperature
config = ising_gibbs(n=20, beta=0.44, n_steps=200000)
```


6.4 Perfect Simulation: Motivation and CFTP I

The problem with standard MCMC. The Markov chain (X_n) only *approximates* π after n steps: the distribution of X_n is μP^n , which is close to π but not identical. We never know exactly how close μP^n is to π without knowing the mixing time (which may itself be hard to compute). In practice, choosing an adequate burn-in requires additional analysis or heuristics, and there is always some residual bias.

Perfect Simulation (Definition)

A **perfect simulation** algorithm generates a sample that has distribution *exactly* π — not approximately, but with zero error — in a finite (but random) amount of time, and without needing to know the mixing time in advance.

The key idea, due to Propp and Wilson (1996), is **Coupling from the Past (CFTP)**. The word “coupling” refers to running multiple chains simultaneously using the same sequence of random numbers. “From the Past” refers to the crucial trick of running chains *backward* in time.

6.4 Perfect Simulation: Motivation and CFTP II

The CFTP idea. Instead of running a single chain forward from time 0 to time R (starting from one initial state and hoping the chain has mixed), CFTP runs chains *backward* from time $-N$ to time 0, starting simultaneously from *every possible initial state*, using the *same* sequence of random variables $U_{-N}, \dots, U_{-1}, U_0$.

If all these chains (one per initial state) end up in the same state Z at time 0, then two things are true: (1) Z is the same regardless of where we started at time $-N$, (2) Since the chain would have mixed by time 0 no matter where it started (by the ergodic theorem), the output $Z \sim \pi$ exactly.

6.4 Perfect Simulation: Motivation and CFTP III

Algorithm 49 (CFTP — General Version)

Require: State space $\mathcal{S}$, ergodic chain (X_n) with update rule φ .

- ① Set $n = 1$.
- ② Simulate, starting from every state $s \in \mathcal{S}$ at time $-N_n$, the chain X^s using the *same* i.i.d. U_i for $i = -N_n, \dots, 0$.
- ③ If all chains reach the same state at time 0: output that state and stop.
- ④ Otherwise, set $n = n + 1$, double $N_n = 2N_{n-1}$, and reuse previously generated U_i (extending the sequence further back in time, while keeping the portion from $-N_{n-1}$ to 0 unchanged).

Why Extend Backward and Keep Old Randomness?

The doubling strategy ensures that if N steps were enough for all chains starting at $-N$ to coalesce (but we didn't check far enough back), the next doubling to $-2N$ will cover it. Keeping the same random numbers from $-N$ to 0 means the chains' behaviour from $-N$ to 0 is preserved exactly; we only add a new random segment from $-2N$ to $-N$.

6.4 Perfect Simulation: Motivation and CFTP IV

Running Algorithm 49 naively requires simulating $|\mathcal{S}|$ chains, one per starting state, which is prohibitive for large $\mathcal{S}$. The key efficiency gain comes from *monotone update functions*.

Definition 6.4.3 (Realizable Monotone Chain)

A Markov chain with update function φ is **realizable monotone** with respect to a partial order $\preceq$ on $\mathcal{S}$ if

$$s \preceq s' \implies \varphi(s, u) \preceq \varphi(s', u) \quad \text{for all } u \in [0, 1].$$

In words: if state s is “smaller” than state s' (in the partial order), then after applying the same random update u , the chain from s is still smaller than the chain from s' . Order is preserved by the update.

6.4 Perfect Simulation: Motivation and CFTP V

Notation

A *partial order* $\preceq$ (“precedes or equals”) on $\mathcal{S}$ is a relation that is reflexive ($s \preceq s$), antisymmetric ($s \preceq s'$ and $s' \preceq s$ implies $s = s'$), and transitive ($s \preceq s'$ and $s' \preceq s''$ implies $s \preceq s''$). For product spaces $\mathcal{S} = C^V$, the coordinate-wise order is the natural choice: $\xi \preceq \xi'$ iff $\xi(v) \leq \xi'(v)$ for all $v \in V$. The *minimum element* $s_{\min}$ satisfies $s_{\min} \preceq s$ for all s ; the *maximum element* $s_{\max}$ satisfies $s \preceq s_{\max}$ for all s .

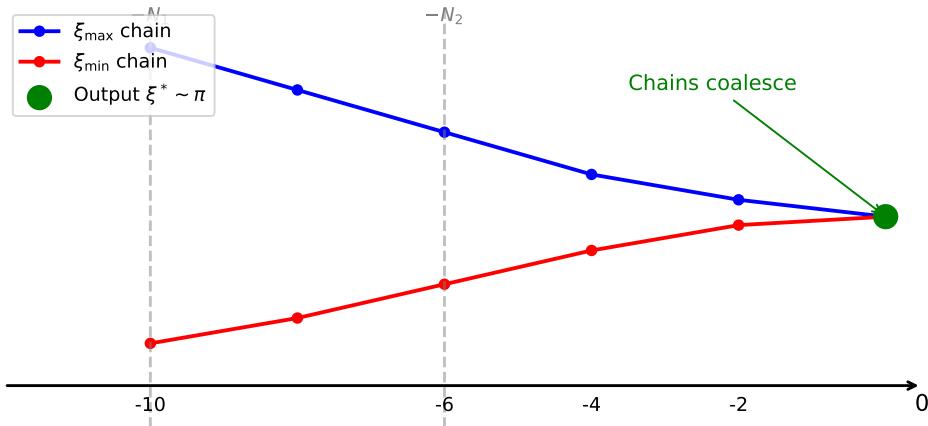
If the chain is realizable monotone, then for any initial states $s_{\min} \preceq s \preceq s_{\max}$, the monotonicity of φ guarantees $\varphi(s_{\min}, u) \preceq \varphi(s, u) \preceq \varphi(s_{\max}, u)$ for all u (and by induction for all subsequent steps). Therefore, if the min-chain and max-chain coalesce (reach the same state), every chain from every starting state must also have coalesced (it is “sandwiched” between them).

Algorithm 50 (Efficient CFTP for Monotone Chains)

Only run *two* chains: one starting from the minimum element $s_{\min}$ and one from the maximum element $s_{\max}$. When these two chains coalesce at time 0 (reach the same state), that common state is output as the exact sample from π . This reduces the simulation cost from $|\mathcal{S}|$ chains to just 2 chains.

6.4 Perfect Simulation: Motivation and CFTP VI

Coupling from the Past (CFTP): schematic



6.4 Perfect Simulation: Motivation and CFTP VII

Theorem 6.4.2 (CFTP Correctness)

Suppose τ_{cftp} (tau-cftp, the coalescence time) is finite with probability 1. Then the output of Algorithm 49 (or Algorithm 50 for monotone chains) has distribution *exactly* π .

Proof sketch. Fix any starting time $-N$ with $N \geq \tau_{\text{cftp}}$ (large enough that all chains have coalesced by time 0). Because all chains starting at time $-N$ from any initial state s reach the same state Z at time 0, Z is determined entirely by the random numbers $U_{-N}, \dots, U_0$ and does not depend on the starting state. By the ergodic theorem, the chain starting from any s at time $-N$ has distribution $\approx \pi$ at time 0 (for large N). Since Z is the same regardless of s , its distribution must be exactly π . $\square$

Proposition 6.4.5 (CFTP Terminates)

If the chain is realizable monotone with an ergodic update function, Algorithm 50 terminates with probability 1. The expected number of backward steps needed is at most a constant times the mixing time $\tau(1/4)$.

6.4 Monotone and Anti-monotone Distributions I

Definition 6.4.9 (Monotone Distribution)

A distribution π on $\mathcal{S} = C^V$ is **monotone** with respect to the coordinate-wise order $\preceq$ if, for all $\xi \preceq \xi'$ and all $v \in V$:

$$P(Z(v) \leq c \mid Z_{-v} = \xi_{-v}) \geq P(Z(v) \leq c \mid Z_{-v} = \xi'_{-v}) \quad \text{for all } c \in C.$$

Intuitively: a lower surrounding configuration (smaller ξ_{-v}) makes lower values at v *more* likely; starting from a higher surrounding configuration makes higher values more likely.

Lemma 6.4.10

If π is monotone, then the Gibbs update function is a monotone update function (in the sense of Definition 6.4.3). Therefore the Gibbs chain for a monotone distribution has a realizable monotone update, and the efficient CFTP Algorithm 50 applies directly.

6.4 Monotone and Anti-monotone Distributions II

Example 6.4.12 (Generalised Hard-core Model). The hard-core model is actually *anti-monotone*: if $\xi \preceq \xi'$ (fewer occupied vertices in ξ than ξ'), then in the lower configuration ξ , fewer neighbours of a given vertex v are occupied, meaning v is *more* likely to be occupied in ξ than in ξ' . Higher surrounding configurations inhibit occupation of v , so conditional probabilities are *decreasing* in the surrounding configuration.

6.4 Monotone and Anti-monotone Distributions III

Definition 6.4.11 (Anti-monotone Distribution)

A distribution π on $\mathcal{S} = C^V$ is **anti-monotone** if for $\xi \preceq \xi'$:

$$P(Z(v) \leq c \mid Z_{-v} = \xi_{-v}) \leq P(Z(v) \leq c \mid Z_{-v} = \xi'_{-v}).$$

That is, higher surrounding configurations make lower values at v more likely (the opposite of monotone).

Example 6.4.13: Ising model. For $\beta > 0$ (ferromagnet): a higher surrounding spin sum (more +1 neighbours) makes +1 at v more likely — so $\alpha(\xi, v)$ is *increasing* in ξ . The ferromagnetic Ising model is *monotone*, and Algorithm 50 applies.

For $\beta < 0$ (anti-ferromagnet): more +1 neighbours make +1 at v *less* likely (opposing spins are preferred) — so $\alpha(\xi, v)$ is *decreasing* in ξ . The anti-ferromagnetic Ising model is *anti-monotone*. For anti-monotone distributions, the efficient CFTP must track a *bivariate* chain $(\xi_{\min}, \xi_{\max})$ with a modified update that swaps the roles of min and max at each step (because a step that increases one chain decreases the other under anti-monotonicity).

6.4 Monotone and Anti-monotone Distributions IV

Proposition 6.4.17

If π is anti-monotone, the modified CFTP Algorithm 51 terminates with probability 1 and outputs a sample from π exactly, using only two chains (the min and max under the coordinate-wise order).

6.4 CFTP: Python Sketch for Hard-core Model

```
import numpy as np

def cftp_hardcore(G, lam, max_doublings=20, seed=0):
    """
    CFTP (monotone version) for hard-core model.
    G : dict {vertex: [neighbor_list]}
    lam : activity parameter lambda > 0
    Returns an exact sample from the hard-core distribution.
    """
    np.random.seed(seed)
    vertices = list(G.keys())
    n = len(vertices)
    prob_occ = lam / (1 + lam)  # P(occupy / all neighbors empty)

    for doubling in range(max_doublings):
        N = 2 ** doubling  # number of backward steps
        # Pre-generate: which vertex to update and acceptance coin
        vtx_idx = np.random.randint(0, n, size=N)
        coins = np.random.rand(N)

        # Min chain: start from all-empty (xi = 0 everywhere)
        xi_min = {v: 0 for v in vertices}
        # Max chain: start from all-occupied if feasible
        xi_max = {v: 1 for v in vertices}

        for k in range(N):
            v = vertices[vtx_idx[k]]
```

6.5 Approximate Counting: The Problem I

Many important combinatorial problems ask: *how many* feasible configurations exist? Computing this count exactly is often computationally intractable. For example:

- How many proper r -colourings does a graph G have? (Answer: $z_G = |\Lambda|$, the number of valid colourings.)
- How many independent sets are in G ?
- How many binary vectors $\xi \in \{0, 1\}^d$ satisfy a given knapsack constraint?

Computing z_G exactly is #P-complete for most such problems (a complexity class even harder than NP). However, we can *approximate* it efficiently using MCMC.

Definition: (ε, δ) -Approximation Scheme (FPRASC)

An estimator $\hat{z}_G$ is a **fully polynomial randomised approximation scheme** (FPRASC, pronounced “fp-ras”) if

$$\mathbb{P}(|\hat{z}_G - z_G| > \varepsilon z_G) \leq \delta,$$

and its running time is polynomial in $1/\varepsilon$ (epsilon, the relative accuracy), $\ln(1/\delta)$ (delta, the failure probability), and the problem size.

6.5 Approximate Counting: The Problem II

Notation

ε (epsilon, “ep-si-lon”) is the desired relative accuracy: we want $\hat{z}_G$ to be within a factor $(1 \pm \varepsilon)$ of the true count z_G . δ (delta, “del-ta”) is the allowed probability of failure: the estimate fails to be accurate only with probability at most δ . The condition “polynomial in $1/\varepsilon$, $\ln(1/\delta)$, and problem size” means the algorithm is efficient: doubling the required accuracy only polynomially increases the running time.

6.5 Approximate Counting: The Problem III

Why the naive approach fails. One might estimate $z_G/2^M$ (the fraction of all 2^M possible configurations that are feasible) by sampling R configurations uniformly at random and checking feasibility. But $z_G/2^M$ can be exponentially small (e.g., for dense graphs, almost no assignment is a valid colouring), requiring exponentially many samples to get even one feasible configuration. The solution is a clever *telescoping* (also called *self-reducibility*) decomposition.

6.5 Approximate Counting: The Problem IV

Telescoping / Self-Reducibility Approach

Decompose the count as a product of near-1 ratios:

$$z_G = z_0 \cdot \frac{z_1}{z_0} \cdot \frac{z_2}{z_1} \cdots \frac{z_m}{z_{m-1}}.$$

Here:

- $z_0 = 2^M$ (or r^M) is known exactly (the count for a trivial base problem),
- $z_m = z_G$ is the target count (unknown),
- each ratio $\varrho_i = z_i/z_{i-1}$ is bounded: $\varrho_i \geq 1/2$ (so no single step changes the count by more than a factor of 2),
- each ratio ϱ_i can be estimated efficiently via MCMC, because $\varrho_i = \mathbb{P}_{\pi(i-1)}(\text{configuration is still feasible in } G_i)$, which is just the expectation of an indicator function under the chain from step $i - 1$.

6.5 Approximate Counting: The Problem V

How to Read the Telescoping Product

The product $\prod_{i=1}^m (z_i/z_{i-1}) = z_m/z_0$ just says that the product of ratios telescopes to the total ratio z_G/z_0 . Multiplying both sides by z_0 : $z_G = z_0 \prod_{i=1}^m \varrho_i$. We know z_0 , and we estimate each ϱ_i separately. Since each $\varrho_i \in [1/2, 1]$, estimating each one is much easier than estimating the tiny fraction z_G/z_0 directly.

6.5 Approximate Counting: The Problem VI

$$Z_G = Z_0 \times \frac{Z_1}{Z_0} \times \frac{Z_2}{Z_1} \times \cdots \times \frac{Z_m}{Z_{m-1}}$$

$$= 2^M \cdot \varrho_1 \cdot \varrho_2 \cdots \varrho_m$$

where $\varrho_i = z_i/z_{i-1} \geq 1/2$ ensures efficient estimation

6.5.1 FPRAS for Independent Sets I

Setup. Let $G = (V, E)$ with $M = |V|$ vertices and $m = |E|$ edges. Order the edges $E = \{e_1, \dots, e_m\}$ and define the sequence of graphs $G_0, G_1, \dots, G_m$ where $G_0 = (V, \emptyset)$ is the edgeless graph (no edges) and $G_i = (V, \{e_1, \dots, e_i\})$ adds one edge at a time. Define Λ_i as the set of independent sets of G_i and $z_i = |\Lambda_i|$.

At G_0 : every subset of V is trivially an independent set (no edges to violate), so $z_0 = 2^M$. At $G_m = G$: $z_m = z_G$ is the unknown count of independent sets in the original graph. Each step adds one edge, potentially removing some independent sets from Λ_{i-1} to get Λ_i .

6.5.1 FPRAS for Independent Sets II

Key Lemma 6.5.7: Each Ratio is at Least $1/2$

For $1 \leq i \leq m$: $\varrho_i = z_i/z_{i-1} \geq 1/2$.

Proof. Adding edge $e_i = \{u, v\}$ removes from Λ_{i-1} exactly those independent sets that contain *both* u and v . Let A = set of independent sets in Λ_{i-1} containing both u and v . Consider the map $\phi : A \rightarrow \Lambda_{i-1} \setminus A$ defined by $\phi(\xi) = \xi(v \rightarrow 0)$ (remove v from ξ). This map is an injection: if $\xi \in A$ then $\phi(\xi)$ is a valid independent set in G_{i-1} (removing a vertex cannot create a conflict) and $\phi(\xi) \notin A$ (vertex v is now 0). Since ϕ is injective from A into $\Lambda_{i-1} \setminus A$, we have $|A| \leq |\Lambda_{i-1} \setminus A| = z_{i-1} - |A|$, giving $2|A| \leq z_{i-1}$, i.e. $|A| \leq z_{i-1}/2$. Therefore $z_i = z_{i-1} - |A| \geq z_{i-1} - z_{i-1}/2 = z_{i-1}/2$. Hence $\varrho_i \geq 1/2$. $\square$

6.5.1 FPRAS for Independent Sets III

Algorithm 54: Estimating a Single Ratio ϱ_i

Require: Graphs G_{i-1} , G_i ; a rapidly mixing MCMC chain with stationary distribution $\pi^{(i-1)}$ (uniform on Λ_{i-1}); accuracy parameters ε' and δ' .

- ① Set $R = \frac{200m^2}{\varepsilon^2} \ln\left(\frac{2m}{\delta}\right)$ (number of independent samples needed).
- ② For $j = 1, \dots, R$:
 - ① Run the MCMC chain for $\tau(\varepsilon/(16m))$ steps (enough to get an approximately stationary sample ξ from $\pi^{(i-1)}$).
 - ② Set $\tilde{Y}_j = \mathbf{1}(\xi \in \Lambda_{G_i})$ (is ξ still a valid independent set in G_i , which has the extra edge e_i ?).
- ③ Return $\tilde{\varrho}_i = \frac{1}{R} \sum_{j=1}^R \tilde{Y}_j$ (the sample fraction of G_{i-1} -independent sets that are also G_i -independent sets).

6.5.1 FPRAS for Independent Sets IV

Notation for Algorithm 54

The estimator $\tilde{\varrho}_i$ (“rho-tilde sub i”) is the average of indicator random variables $\tilde{Y}_j$. Since $\mathbb{E}[\tilde{Y}_j] = \mathbb{P}_{\pi^{(i-1)}}(\xi \in \Lambda_{G_i}) = z_i/z_{i-1} = \varrho_i$, the estimator is (approximately) unbiased. The sample size R is chosen large enough to ensure the estimate is accurate with high probability.

Algorithm 55: (ε, δ) -FPRASC for z_G

- 1 For each $i = 1, \dots, m$: compute $\tilde{\varrho}_i$ using Algorithm 54.
- 2 Return $\hat{z}_G = 2^M \prod_{i=1}^m \tilde{\varrho}_i$.

6.5.1 FPRAS for Independent Sets V

Proposition 6.5.4 (FPRASC Running Time)

Let the MCMC chain be rapidly mixing with mixing time $\tau(\kappa)$. Then Algorithm 55 is an (ε, δ) -FPRASC for z_G with total running time

$$\frac{200 m^3}{\varepsilon^2} \ln\left(\frac{2m}{\delta}\right) \tau\left(\frac{\varepsilon}{16m}\right).$$

This is polynomial in m (the number of edges), $1/\varepsilon$, $\ln(1/\delta)$, and the mixing time τ .

Notation

The factor m^3 comes from m ratios, each estimated with $R \propto m^2/\varepsilon^2$ samples. The mixing time argument $\varepsilon/(16m)$ ensures each individual sample is close enough to stationarity that the ratio estimates are accurate.

Why the error analysis works. Each ratio ϱ_i is estimated with relative error $\leq \varepsilon/(2m)$ and failure probability $\leq \delta/m$. By the *union bound* (the probability of at least one of m bad events is at most m times the probability of a single bad event), all m estimates are simultaneously good with probability

6.5.1 FPRAS for Independent Sets VI

$\geq 1 - m \cdot (\delta/m) = 1 - \delta$. A standard calculation shows that when all individual estimates are accurate to relative error $\varepsilon/(2m)$, the product estimate satisfies $|\hat{z}_G/z_G - 1| \leq \varepsilon$ with probability $\geq 1 - \delta$.

6.5.2 Graph Colourings and Knapsack FPRAS I

The same telescoping framework applies to graph colourings and 0-1 knapsack problems.

For graph colourings. Order the edges $E = \{e_1, \dots, e_m\}$ and define z_i as the number of proper r -colourings of $G_i = (V, \{e_1, \dots, e_i\})$. The base case $z_0 = r^M$ (all r^M colour assignments are valid on the edgeless graph) is known exactly. Each step adds one edge, reducing the number of valid colourings.

Lemma 6.5.14

If $r > 2\Delta^2$ (where Δ is the maximum degree of G), then $\varrho_i = z_i/z_{i-1} \geq 1/2$ for all i .

Proof sketch. Adding edge $e_i = \{u, v\}$ removes colourings where $\xi(u) = \xi(v)$ (same colour on a newly connected pair). For a random proper colouring ξ of G_{i-1} , the probability that $\xi(u) = \xi(v)$ is at most $1/(r - \Delta)$, since vertex v has at most Δ neighbours (in G_{i-1}) that restrict its colour. Under the condition $r > 2\Delta^2$: $1/(r - \Delta) < 1/(2\Delta^2 - \Delta) \leq 1/2$ for $\Delta \geq 1$. So $\varrho_i = 1 - P(\xi(u) = \xi(v)) \geq 1 - 1/2 = 1/2$. $\square$

6.5.2 Graph Colourings and Knapsack FPRAS II

Lemma 6.5.16 (FPRASC for Colourings)

If $r > 2\Delta^2$, there exists an (ε, δ) -FPRASC for the number of proper r -colourings with running time

$$\frac{200 m^3}{\varepsilon^2} \ln\left(\frac{2m}{\delta}\right) \cdot \frac{r}{r - 2\Delta} \cdot M \ln\left(\frac{16mM}{\varepsilon}\right),$$

where the mixing time bound $\tau(\kappa) \leq \frac{r}{r-2\Delta} M \ln(M/\kappa)$ from Lemma 6.5.15 was substituted. All factors are polynomial in M , m , $1/\varepsilon$, and $\ln(1/\delta)$.

For the 0-1 knapsack problem. Given d items with weights $a_1, \dots, a_d$ (positive integers) and capacity b , the feasible set is $\Lambda = \{\xi \in \{0, 1\}^d : \sum_{i=1}^d a_i \xi_i \leq b\}$ (all subsets of items whose total weight does not exceed b). We want to count $z = |\Lambda|$.

A Gibbs-like sampler is defined on a sequence of subproblems $\Lambda_0 \supseteq \Lambda_1 \supseteq \dots \supseteq \Lambda_m = \Lambda$, where each step tightens the capacity constraint. Each ratio z_i/z_{i-1} can be estimated by a MCMC chain.

6.5.2 Graph Colourings and Knapsack FPRAS III

Lemma 6.5.18 (Dyer, Frieze & Kannan)

The Gibbs sampler for the 0-1 knapsack is rapidly mixing with mixing time $O(d^{9/2+\kappa})$ for any $\kappa > 0$. This gives an (ε, δ) -FPRASC for $|\Lambda|$ with running time polynomial in d , $1/\varepsilon$, and $\ln(1/\delta)$.

6.6 Motif Finding in DNA Sequences I

A *motif* (from French: “pattern”) in molecular biology is a short, conserved sequence pattern that recurs across a collection of biological sequences, usually because it plays an important functional role. For example, a transcription factor binding site — a specific DNA sequence that a protein “reads” to regulate gene expression — appears at similar (but not identical) positions in the regulatory regions of co-regulated genes. Finding such motifs computationally is a fundamental problem in bioinformatics.

6.6 Motif Finding in DNA Sequences II

Model Setup

- There are k DNA sequences (“ k ”), each of length n , over the alphabet $\Sigma = \{A, C, G, T\}$ (the four DNA nucleotides, encoded as $\{1, 2, 3, 4\}$, so $d = |\Sigma| = 4$).
- The *motif* has a fixed length w (“ w ” for “width”). In sequence i , the motif starts at an *unknown* position $\xi_i^\bullet \in \{1, \dots, n - w + 1\}$ (“ ξ - i star”).
- Letters *outside* the motif are generated i.i.d. from the *background distribution* $\theta_0 = (\theta_{1,0}, \dots, \theta_{d,0})^\top$ (a probability vector over the alphabet).
- Letters at column j ($j = 1, \dots, w$) of the motif are generated from the *motif distribution* $\theta_j = (\theta_{1,j}, \dots, \theta_{d,j})^\top$ (a possibly different probability vector for each motif column).
- The *position weight matrix* (PWM) is $\Theta = (\theta_0, \theta_1, \dots, \theta_w)$ (theta, “thay-ta”).

6.6 Motif Finding in DNA Sequences III

Given: The observed sequences $\mathbf{R} = (r_{ij})_{1 \leq i \leq k, 1 \leq j \leq n}$ (the actual DNA letters) and the PWM Θ (assumed known or estimated separately). **Find:** The starting positions $\xi = (\xi_1, \dots, \xi_k)$ that maximise the likelihood (the probability of observing the sequences given those starting positions).

For a configuration $\xi = (\xi_1, \dots, \xi_k)$, define:

- $\mathbf{A}_\xi$: the $k \times w$ matrix whose row i is the w -letter segment of sequence i starting at position ξ_i .
- $\mathbf{C}_\xi$: the $d \times w$ count matrix; $\mathbf{C}_\xi(l, j)$ is the number of sequences i where the letter at motif column j is l .
- $\mathbf{D}_\xi$: the d -vector of background letter counts (letters not in any motif segment).

The complete-data likelihood factors as a product over background letters and motif columns:

$$f(\xi) = \underbrace{\prod_{l=1}^d \theta_{l,0}^{\mathbf{D}_\xi(l)}}_{\text{background contribution}} \times \underbrace{\prod_{j=1}^w \prod_{l=1}^d \theta_{l,j}^{\mathbf{C}_\xi(l,j)}}_{\text{motif contribution}}.$$

6.6 Motif Finding in DNA Sequences IV

Notation

$\theta_{l,0}^{\mathbf{D}_\xi(l)}$ (“theta-l-zero to the power D-xi-l”) is the probability of seeing letter l exactly $\mathbf{D}_\xi(l)$ times in the background, under the background model. $\theta_{l,j}^{\mathbf{C}_\xi(l,j)}$ similarly gives the probability of the observed count of letter l at motif column j . The product over all l and j gives the total probability of the data given the starting positions ξ .

6.6 Motif Finding in DNA Sequences V

To handle the combinatorial optimisation (finding the ξ that maximises f) using MCMC, we adopt a probabilistic approach via the **Gibbs distribution**:

$$\pi_{\xi} = \frac{1}{z(T)} f(\xi)^{1/T} = \frac{\exp(\frac{1}{T} \log f(\xi))}{z(T)}.$$

Here $T > 0$ (“T” for temperature) controls the concentration of the distribution.

Notation and Interpretation of the Temperature T

When $T = 1$: $\pi_{\xi} \propto f(\xi)$ is proportional to the likelihood itself. When $T \rightarrow 0$ (cooling): π_{ξ} concentrates entirely on the maximisers of f (the starting positions that make the observed sequences most probable under the model). When $T \rightarrow \infty$ (heating): π_{ξ} approaches the uniform distribution (all starting positions equally likely). Running a Gibbs sampler on π while slowly decreasing T from large to small implements *simulated annealing*: at high T the chain mixes fast and explores widely; at low T it concentrates on the optimum.

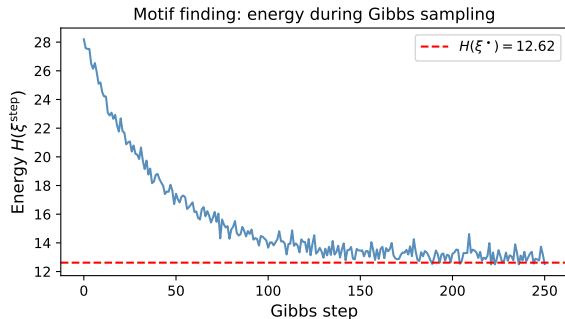
6.6 Motif Finding in DNA Sequences VI

Gibbs Sampler for Motif Finding (Algorithm 58)

Given: Current configuration $\xi = (\xi_1, \dots, \xi_k)$; temperature T .

- 1 Choose row $v \in \{1, \dots, k\}$ (one DNA sequence) uniformly at random.
- 2 For each candidate starting position $s \in \{1, \dots, n - w + 1\}$, compute the unnormalised conditional weight $\tilde{p}(s) = f(\xi(v \rightarrow s))^{1/T}$ (the likelihood if sequence v starts at position s).
- 3 Normalise: $p(s) = \tilde{p}(s) / \sum_{r=1}^{n-w+1} \tilde{p}(r)$.
- 4 Sample the new starting position $s^* \sim p(\cdot)$ and update $\xi_v \leftarrow s^*$.

6.6 Motif Finding in DNA Sequences VII



The key computational simplification that makes each Gibbs step fast: the ratio $\pi_{\xi(v \rightarrow s)} / \pi_{\xi(v \rightarrow r)}$ depends only on the *changed row* of $\mathbf{A}_\xi$ (row v), not on the entire configuration. This reduces each Gibbs step from $O(k \cdot n \cdot w)$ (recomputing everything) to $O(n \cdot w)$ (updating only the changed row), making the algorithm practical for biological datasets.

Numerical example (Section 6.6.3).

Parameters: $k = 30$ sequences, $n = 20$, $w = 13$. True starting positions ξ^* simulated from the model. The energy (negative log-likelihood) of the true solution is $H(\xi^*) = -\log f(\xi^*) = 12.62$.

After just 200 Gibbs steps: the estimated energy is $H(\xi^{(200)}) = 12.64$. Only 2 of the 30 starting positions differ from ξ^* (by at most 1 position each). The Gibbs sampler recovers the true motif positions very reliably and quickly.

6.7 MCMC Estimation: Ergodic Theorem and CLT I

We now return to the fundamental statistical question: given an ergodic Markov chain with stationary distribution π , how do we use it to estimate the quantity

$$I = \mathbb{E}_{\pi} f(X) = \sum_{s \in \mathcal{S}} \pi_s f(s)?$$

Here $\mathbb{E}_{\pi}$ (“expectation under π ”) denotes the expected value when the random state X is drawn from distribution π , and $f : \mathcal{S} \rightarrow \mathbb{R}$ is any real-valued function of interest.

The MCMC estimator is simply the sample average of f over the chain:

$$\hat{Y}_R = \frac{1}{R} \sum_{j=1}^R f(X_j),$$

where $(X_j)_{j \geq 0}$ is the ergodic chain and $Y_j = f(X_j)$ are the observed values.

6.7 MCMC Estimation: Ergodic Theorem and CLT II

Proposition 6.7.1 (Strong Law / Ergodic Theorem for Markov Chains)

If (X_n) is ergodic (irreducible and aperiodic with finite state space) and $f : \mathcal{S} \rightarrow \mathbb{R}$ is any real-valued function, then with probability 1 (i.e., almost surely):

$$\hat{Y}_R = \frac{1}{R} \sum_{j=1}^R f(X_j) \xrightarrow{R \rightarrow \infty} I = \sum_{s \in \mathcal{S}} \pi_s f(s).$$

This convergence holds regardless of the initial distribution μ (“mu”) of X_0 .

6.7 MCMC Estimation: Ergodic Theorem and CLT III

How to Read the Ergodic Theorem

This is the Markov chain analogue of the classical law of large numbers. Just as the sample average of i.i.d. random variables converges to the population mean, the time average of a function of an ergodic Markov chain converges to its expectation under the stationary distribution. The crucial difference: the chain samples are *correlated* (each state depends on the previous), which affects how quickly the average converges and how we should estimate its variance.

6.7 MCMC Estimation: Ergodic Theorem and CLT IV

Unlike the i.i.d. case, the samples $Y_1, Y_2, \dots$ are correlated (each $Y_j = f(X_j)$ depends on $Y_{j-1} = f(X_{j-1})$ through the chain transition). This correlation affects the variance of $\hat{Y}_R$.

Proposition 6.7.5 (Central Limit Theorem for Markov Chains)

If (X_n) is ergodic and f is real-valued, then as $R \rightarrow \infty$:

$$\sqrt{R}(\hat{Y}_R - I) \xrightarrow{d} \mathcal{N}(0, \varsigma^2),$$

where ς^2 (“varsigma squared”, or “zeta squared” in some notations) is the **asymptotic variance** (also called the *time-average variance constant*, TAVC) satisfying

$$0 < \varsigma^2 = \lim_{R \rightarrow \infty} R \text{Var}(\hat{Y}_R) < \infty.$$

Both the limit and the CLT hold independently of the initial distribution μ .

6.7 MCMC Estimation: Ergodic Theorem and CLT V

Notation

$\xrightarrow{d}$ means “converges in distribution” (the distribution of $\sqrt{R}(\hat{Y}_R - I)$ approaches the normal distribution as $R \rightarrow \infty$). $\mathcal{N}(0, \varsigma^2)$ is the normal (Gaussian) distribution with mean 0 and variance ς^2 . $\text{Var}(\hat{Y}_R)$ (“variance of Y-hat-R”) is the expected squared deviation of $\hat{Y}_R$ from I : $\text{Var}(\hat{Y}_R) = \mathbb{E}[(\hat{Y}_R - I)^2]$.

6.7 MCMC Estimation: Ergodic Theorem and CLT VI

The TAVC ς^2 can be expressed in terms of the autocovariances of the stationary sequence. Let $Y_j = f(X_j)$ when $X_0 \sim \pi$ (so the chain is in stationarity), and define the *lag- k autocovariance*:

$$\rho_k = \text{Cov}(Y_0, Y_k) = \mathbb{E}[(Y_0 - I)(Y_k - I)], \quad k = 0, 1, 2, \dots$$

Lemma 6.7.9 (Variance Formula)

If $\mathbf{P}$ is regular, then

$$\text{Var}(\hat{Y}_R) = \frac{1}{R} \left(1 + 2 \sum_{j=1}^{R-1} \left(1 - \frac{j}{R} \right) \rho_j \right) \text{Var}(Y_0),$$

so that the TAVC satisfies

$$\varsigma^2 = \lim_{R \rightarrow \infty} R \text{Var}(\hat{Y}_R) = \text{Var}(Y_0) + 2 \sum_{k=1}^{\infty} \rho_k = \rho_0 + 2 \sum_{k=1}^{\infty} \rho_k.$$

6.7 MCMC Estimation: Ergodic Theorem and CLT VII

Notation

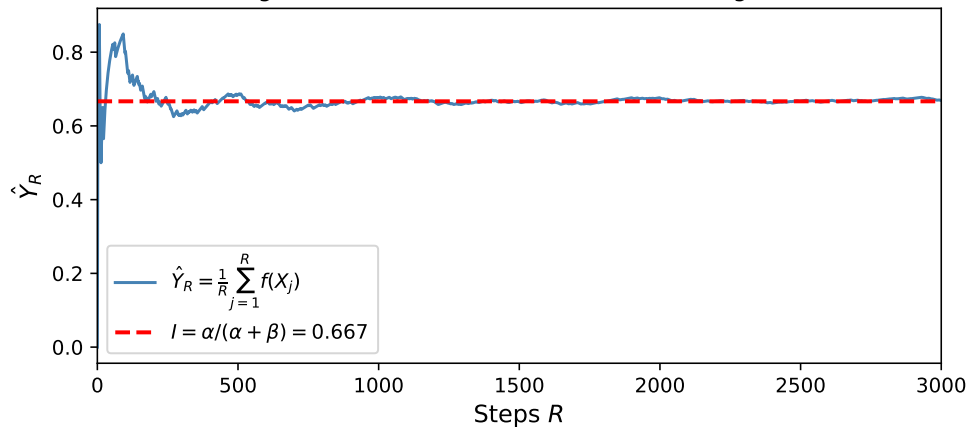
ρ_k (rho, “ro”, sub k) is the covariance between Y_0 and Y_k (how correlated are the values k steps apart?). $\rho_0 = \text{Var}(Y_0)$ is the variance of a single observation. The sum $\varsigma^2 = \rho_0 + 2 \sum_{k=1}^{\infty} \rho_k$ is called the *spectral density at frequency zero*.

How to Read $\varsigma^2 = \rho_0 + 2 \sum_{k \geq 1} \rho_k$

If the chain is fast-mixing, the autocovariances ρ_k decay quickly to zero, and $\varsigma^2 \approx \rho_0$ (close to the i.i.d. variance). If the chain is slow-mixing, ρ_k stays positive for many lags, and $\varsigma^2 \gg \rho_0$: MCMC samples are much less informative than i.i.d. samples. When $\rho_k < 0$ for large k (negative autocorrelation, possible with some specialised samplers), $\varsigma^2 < \rho_0$ and MCMC can actually be *more* efficient than i.i.d. sampling.

6.7 MCMC Estimation: Ergodic Theorem and CLT VIII

Ergodic theorem: MCMC estimator convergence



6.7 Regenerative Structure and Asymptotic Variance I

A key tool for analysing the TAVC is the *regenerative structure* of ergodic Markov chains.

Regeneration times. Fix a reference state $s_0 \in \mathcal{S}$ (any state will do, though choosing a frequently-visited state is efficient) and let $\gamma_0 < \gamma_1 < \gamma_2 < \dots$ be the successive times at which the chain visits s_0 : $\gamma_l = \min\{n > \gamma_{l-1} : X_n = s_0\}$. At each return to s_0 , the chain *regenerates*: its future evolution is independent of its past. The intervals $[\gamma_{l-1}, \gamma_l)$ are called *cycles*.

The *cycle lengths* $\eta_l = \gamma_l - \gamma_{l-1}$ (eta, “ay-ta”) and the *cycle sums* $W_l = \sum_{j=\gamma_{l-1}}^{\gamma_l-1} f(X_j)$ (total of f over cycle l) are i.i.d. sequences (by the strong Markov property at regeneration times).

6.7 Regenerative Structure and Asymptotic Variance II

Representation as a Ratio of i.i.d. Means

The ergodic theorem and the CLT for ratios of i.i.d. sums give:

$$I = \lim_{R \rightarrow \infty} \hat{Y}_R = \frac{\mathbb{E}[W_1]}{\mathbb{E}[\eta_1]},$$

and the TAVC satisfies

$$\varsigma^2 = \frac{\text{Var}(W_1 - I \eta_1)}{\mathbb{E}[\eta_1]}.$$

This is the *regenerative ratio estimator* formula.

6.7 Regenerative Structure and Asymptotic Variance III

Notation

$\mathbb{E}[W_1]$ is the expected sum of $f(X_j)$ over one regenerative cycle. $\mathbb{E}[\eta_1]$ is the expected length of one cycle; this equals $1/\pi_{s_0}$ (the reciprocal of the stationary probability of the reference state).

$\text{Var}(W_1 - I \eta_1)$ is the variance of the “centred cycle sum”: the cycle sum minus I times the cycle length (a bias correction).

6.7 Regenerative Structure and Asymptotic Variance IV

Example 6.7.12: Two-State Chain

Consider $\mathcal{S} = \{0, 1\}$ with $\mathbf{P} = \begin{pmatrix} 1 - \alpha & \alpha \\ \beta & 1 - \beta \end{pmatrix}$ (where α and β are as defined earlier), $f(x) = x$ (so $I = \mathbb{E}_\pi[X] = \pi_1 = \alpha/(\alpha + \beta)$), and reference state $s_0 = 0$.

The true expected value is $I = \alpha/(\alpha + \beta)$.

The expected cycle length is $\mathbb{E}[\eta_1] = 1/\pi_0 = (\alpha + \beta)/\beta$ (the expected return time to state 0).

A standard calculation gives $\mathbb{E}[W_1] = \alpha/\beta$ and $\text{Var}(W_1 - I\eta_1) = \alpha\beta(2 - \alpha - \beta)/(\alpha + \beta)^2$.

Plugging into the regenerative formula:

$$\varsigma^2 = \frac{\text{Var}(W_1 - I\eta_1)}{\mathbb{E}[\eta_1]} = \frac{\alpha\beta(2 - \alpha - \beta)/(\alpha + \beta)^2}{(\alpha + \beta)/\beta} = \frac{\alpha\beta(2 - \alpha - \beta)}{(\alpha + \beta)^3}.$$

Comparison with i.i.d. sampling: if the Y_j were independent, $\text{Var}(Y^{\text{ind}}) = \pi_0\pi_1 = \alpha\beta/(\alpha + \beta)^2$, so:

$$\frac{\varsigma^2}{\text{Var}(Y^{\text{ind}})} = \frac{2 - \alpha - \beta}{\alpha + \beta}.$$

This ratio is less than 1 (MCMC is more efficient than i.i.d.) when $\alpha + \beta > 1$ (the chain mixes very fast — both transition probabilities are large so the chain moves freely between states). It is greater than 1

6.7 Variance Estimation and Confidence Intervals I

In practice, ς^2 (the TAVC) is unknown and must be estimated from the observed chain $(Y_0, Y_1, \dots, Y_{R-1})$ (after discarding burn-in). There are two main estimation methods.

Method 1: Lag Covariance Estimation

Given post-burn-in values $Y_0, \dots, Y_{R-1}$, estimate the lag- k autocovariance by:

$$\hat{\rho}_k = \frac{1}{R-k} \sum_{j=0}^{R-k-1} (Y_j - \hat{Y}_R)(Y_{j+k} - \hat{Y}_R), \quad k = 0, 1, \dots, N,$$

then estimate the TAVC by summing:

$$\hat{\varsigma}_N^2 = \hat{\rho}_0 + 2 \sum_{k=1}^N \hat{\rho}_k$$

for a chosen truncation lag N (typically $N \approx \sqrt{R}$ or chosen by an automatic rule such as stopping when $\hat{\rho}_k$ drops below a threshold).

6.7 Variance Estimation and Confidence Intervals II

Notation

$\hat{\rho}_k$ ("rho-hat sub k") is the estimated lag- k autocovariance: it measures how correlated Y_j and Y_{j+k} are on average. $(Y_j - \hat{Y}_R)(Y_{j+k} - \hat{Y}_R)$ is the product of centred values at lags j and $j+k$. The truncation at lag N avoids estimating very small and noisy autocovariances at large lags. A good rule: include enough lags that the estimated $\hat{\rho}_k$ has decayed to near zero.

6.7 Variance Estimation and Confidence Intervals III

Method 2: Batch Means

Divide the R post-burn-in values $Y_0, \dots, Y_{R-1}$ into m non-overlapping batches of size $n = R/m$. The k -th batch mean is

$$\hat{Y}_k(n) = \frac{1}{n} \sum_{i=(k-1)n}^{kn-1} Y_i, \quad k = 1, \dots, m.$$

For batch size n large enough (much larger than the mixing time), the batch means $\hat{Y}_1(n), \dots, \hat{Y}_m(n)$ are approximately independent and approximately normally distributed with mean I and variance ζ^2/n . The sample variance of the batch means estimates ζ^2/n :

$$V_m^2(n) = \frac{1}{m-1} \sum_{k=1}^m (\hat{Y}_k(n) - \bar{\hat{Y}})^2.$$

A **95% confidence interval** for I is then:

$$\hat{Y}_R \pm t_{m-1, 0.975} \cdot \sqrt{\frac{V_m^2(n)}{m}},$$

6.7 MCMC Estimation: Python Implementation

```
import numpy as np
from scipy import stats

def mcmc_estimate_with_CI(chain_values, burn_in=500, n_batches=20):
    """
    Estimate  $I = E[f(X)]$  and build 95% CI using batch means.
    chain_values : array of  $f(X_j)$  from the MCMC chain
    burn_in      : number of initial samples to discard
    n_batches    : number of batches for batch means method
    """
    Y = chain_values[burn_in:]  # discard burn-in
    R = len(Y)
    I_hat = Y.mean()

    # --- Batch Means ---
    batch_size = R // n_batches
    batches = [Y[k*batch_size:(k+1)*batch_size].mean()
               for k in range(n_batches)]
    batch_std = np.std(batches, ddof=1)
    t_crit = stats.t.ppf(0.975, df=n_batches - 1)
    margin = t_crit * batch_std / np.sqrt(n_batches)

    # --- Lag covariance estimate of  $\text{var}\sigma^2$  ---
    N_lag = min(250, R // 4)  # truncation lag
    Y_c = Y - I_hat
    rho_hat = np.array([np.mean(Y_c[:R-k] * Y_c[k:])
                        for k in range(N_lag + 1)])
```

6.7 Hard-core Model: Worked Numerical Example I

We illustrate the full MCMC estimation pipeline on the hard-core model to show all steps concretely.

Setup. Grid graph $G = (\{1, \dots, 8\}^2, E)$ (an 8×8 grid) with $M = 64$ vertices, where each pair of horizontally or vertically adjacent vertices is connected by an edge. Activity parameter $\lambda = 2$ (particles are favoured; $\lambda > 1$). Quantity of interest: $I = \mathbb{E}_\pi m(\xi) = \mathbb{E}_\pi[\text{number of occupied vertices}]$, the expected number of particles at stationarity.

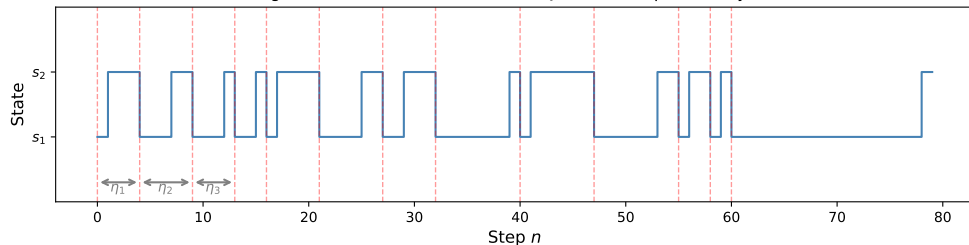
Chain. Gibbs sampler (Algorithm 47) with $R = 10,000$ steps after a burn-in of $k_0 = 500$ steps. The TAVC ζ^2 is estimated by the lag-covariance method with truncation lag $N = 250$.

Results (from Table 6.2 in the book).

- Point estimate: $\hat{Y}_R \approx 24.37$ occupied vertices on average.
- Lag-covariance TAVC estimate: $\hat{\zeta}^2 \approx 8.42$.
- 95% confidence interval: $(23.15, 25.59)$.

6.7 Hard-core Model: Worked Numerical Example II

Regenerative structure: returns to s_0 define independent cycles



Key Takeaway: Effective Sample Size

The TAVC estimate $\hat{\varsigma}^2 \approx 8.42$ is larger than $\rho_0 = \text{Var}(Y_0)$ (the single-sample variance), reflecting positive autocorrelation in the Gibbs chain. The *effective sample size* (ESS) is $\text{ESS} = R \cdot \rho_0 / \hat{\varsigma}^2 \ll R$: the $R = 10,000$ MCMC samples are collectively “worth” only a fraction of that many i.i.d. samples. This happens because the Gibbs chain mixes slowly near the phase transition ($\lambda \approx \lambda_c$ for the hard-core model on a grid), and consecutive samples are highly correlated.

6.8 Stable Simulation: Estimating $\pi(A)$ Exactly I

In some applications we want to estimate the stationary probability $\pi(A) = \sum_{s \in A} \pi_s$ for a specific set $A \subseteq \mathcal{S}$, using *exact* (not approximate) samples from π . The naive approach — use MCMC samples and estimate $\hat{\pi}(A) = \frac{1}{R} \sum_{j=1}^R \mathbf{1}(X_j \in A)$ — suffers from burn-in bias. The *stable simulation scheme* avoids this problem entirely by using a duality relationship called *Siegmund duality*.

Setting. The state space $\mathcal{S} = \{s_1, \dots, s_m\}$ is partially ordered by $\preceq$ (“precedes or equals”). We want to estimate the *down-set probability*:

$$\pi(\{s_j\}^\downarrow) = \pi(\{s' \in \mathcal{S} : s' \preceq s_j\}),$$

the probability under π of being in any state that is $\preceq s_j$. The symbol $\{s_j\}^\downarrow$ (“s-j down”) denotes the *down-set* of s_j : all states that come below s_j in the partial order.

6.8 Stable Simulation: Estimating $\pi(A)$ Exactly II

Definition 6.8.1 (Siegmund Dual)

The Markov chain (Z_n) with transition matrix $\mathbf{P}_Z$ on the extended state space $\mathcal{S}^* = \{s_0\} \cup \mathcal{S}$ (adding an extra absorbing boundary state s_0) is the **Siegmund dual** of (X_n) with transition matrix $\mathbf{P}_X$ if

$$\mathbf{P}_X^n(s_i, \{s_j\}^\downarrow) = \mathbf{P}_Z^n(s_j, \{s_i\}^\uparrow) \quad \forall s_i, s_j \in \mathcal{S}, n \geq 0.$$

Here $\{s_i\}^\uparrow = \{s' \in \mathcal{S}^* : s_i \preceq s'\}$ (“s-i up”) is the *up-set* of s_i .

Notation

$\mathbf{P}_X^n(s_i, \{s_j\}^\downarrow)$ is the probability that chain X , starting from s_i , is in the down-set of s_j after n steps.
 $\mathbf{P}_Z^n(s_j, \{s_i\}^\uparrow)$ is the probability that the dual chain Z , starting from s_j , is in the up-set of s_i after n steps. The duality says these two probabilities are always equal for all n .

6.8 Stable Simulation: Estimating $\pi(A)$ Exactly III

Taking $n \rightarrow \infty$ in the duality relation: On the left: $\mathbf{P}_X^n(s_i, \{s_j\}^\downarrow) \rightarrow \pi(\{s_j\}^\downarrow)$ for any fixed s_i (by the ergodic theorem for chain X). On the right: the extended state space $\mathcal{S}^*$ has the state s_0 acting as an absorbing boundary at the “bottom” and s_m at the “top”. As $n \rightarrow \infty$, the dual chain Z is absorbed at one of these boundaries. Therefore:

$$\pi(\{s_j\}^\downarrow) = \mathbb{P}(Z \text{ absorbed at } s_m \mid Z_0 = s_j).$$

This is the key duality formula: the stationary probability of a down-set equals the *absorption probability* of the dual chain at the top boundary s_m .

6.8 Stable Simulation: Estimating $\pi(A)$ Exactly IV

Algorithm 59 (Stable Simulation Scheme)

Require: Chain (X_n) on $\mathcal{S}$; its Siegmund dual (Z_n) on $\mathcal{S}^* = \{s_0\} \cup \mathcal{S}$; target down-set $\{s_j\}^\downarrow$; number of replications R .

- 1 Compute the Siegmund dual transition matrix $\mathbf{P}_Z$ from $\mathbf{P}_X$.
- 2 For $l = 1, \dots, R$: simulate the dual chain $(Z_n^{(l)})$ starting from s_j until it is absorbed at either s_0 or s_m (two absorbing barriers).
- 3 Estimate: $\hat{\pi}(\{s_j\}^\downarrow) = \frac{1}{R} \sum_{l=1}^R \mathbf{1}(Z^{(l)} \text{ absorbed at } s_m)$.

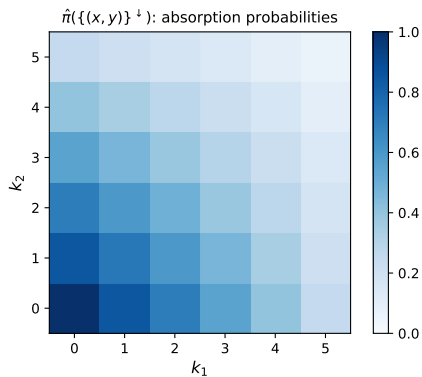
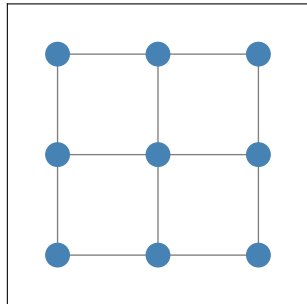
The R replications are i.i.d., so the estimator is exactly unbiased. Setting $R \approx (1.96/b)^2$ gives a 95% confidence interval of half-width b .

Key Advantage over Standard MCMC

Each replication of the dual chain is an *independent* simulation that terminates when absorbed. There is no burn-in, no mixing time to estimate, and no autocorrelation between replications. The estimator is exactly unbiased (not just asymptotically), and standard Binomial confidence intervals apply directly.

6.8 Stable Simulation: Estimating $\pi(A)$ Exactly V

Partial order on $S = \{0, 1, 2\}^2$



6.8 Stable Simulation: Estimating $\pi(A)$ Exactly VI

Definition (Möbius Monotonicity) and Lemma 6.8.2

A chain (X_n) with transition matrix $\mathbf{P}_X$ is **Möbius monotone** (Möbius, pronounced “mo-bius”, named after German mathematician August Möbius) with respect to $\preceq$ if the matrix $\mathbf{C}^{-1}\mathbf{P}_X\mathbf{C} \geq 0$ entry-wise, where $\mathbf{C}(s_i, s_j) = \mathbf{1}(s_i \preceq s_j)$ is the *ordering matrix* (the “ $\preceq$ -incidence matrix”) and $\mathbf{C}^{-1}$ is its inverse, known as the *Möbius function* of the partial order.

A Siegmund dual on $\mathcal{S}^*$ exists if and only if (X_n) is Möbius monotone with respect to $\preceq$.

6.8 Stable Simulation: Estimating $\pi(A)$ Exactly VII

Example 6.8.4: Two-Node Tandem Queue

A two-node tandem queue has $N + 1$ customers that cycle between two servers. The state (x, y) records the number of customers at each server. State space: $\mathcal{S} = \{0, \dots, N\}^2$, partially ordered coordinate-wise. Parameters: $\lambda_1 = 3/16$, $\lambda_2 = 1/16$ (service rates at nodes 1 and 2), $\mu_1 = \mu_2 = 6/16$ (arrival rates), $N = 5$. With $R = 4 \times 10^4$ replications of the dual chain, the down-set probabilities $\hat{\pi}(\{(x, y)\}^\downarrow)$ for all $(x, y) \in \{0, \dots, 5\}^2$ are:

$y \backslash x$	0	1	2	3	4	5
0	0.030	0.081	0.145	0.236	0.351	0.502
1	0.058	0.139	0.249	0.381	0.550	0.763
2	0.079	0.181	0.310	0.468	0.652	0.884
3	0.094	0.207	0.353	0.511	0.710	0.946
4	0.102	0.224	0.372	0.540	0.743	0.981
5	0.111	0.235	0.383	0.558	0.762	1.000

Each entry is an exactly unbiased estimate of $\pi(\{(x, y)\}^\downarrow)$, the probability of being in the down-set of state (x, y) . The entry at $(5, 5) = 1.000$ is correct: every state is $\preceq (5, 5)$ so the probability is exactly 1.

6.9 Bibliographical Notes I

The field of MCMC has a rich history spanning over seven decades, with foundational contributions from physics, statistics, and theoretical computer science.

6.9 Bibliographical Notes II

Foundational References

- **Markov chain theory:** Norris [165] is a clear and accessible introductory textbook on Markov chains; Meyn & Tweedie [147] provide a comprehensive treatment including general (uncountable) state spaces.
- **Metropolis algorithm:** Introduced by Metropolis, Rosenbluth, Rosenbluth, Teller & Teller [145] (1953) for molecular dynamics simulations in physics. Generalised to asymmetric proposals by Hastings [78] (1970).
- **Gibbs sampler:** Introduced by Geman & Geman [62] (1984) for image restoration using simulated annealing on Markov random fields. Popularised in Bayesian statistics by Gelfand & Smith [61] (1990), who recognised its broad applicability for posterior simulation.
- **Coupling from the Past:** Propp & Wilson [170] (1996) introduced the CFTP algorithm and proved its correctness. The efficient monotone version using only min/max chains is due to their work and subsequent development by Huber [85].

6.9 Bibliographical Notes III

FPRAS and Approximate Counting

- The general FPRAS framework via self-reducibility (telescoping): Jerrum, Valiant & Vazirani [100].
- Rapid mixing of the Gibbs sampler for graph colourings with $r \geq 2\Delta + 1$ colours: Jerrum [99] (1995).
- Rapid mixing for 0-1 knapsack problems: Dyer, Frieze & Kannan [43].
- Rapid mixing via the Metropolis algorithm for colourings with $r \geq 2\Delta + 1$: Hayes [85].

Ergodic Theory and Variance Estimation

- Central Limit Theorem for Markov chains: Meyn & Tweedie [147]; Glynn & Iglehart [64].
- Batch means for variance estimation: Damerdjian [30]; Alexopoulos & Seila [4].
- Regenerative simulation: Crane & Iglehart; see also Glynn & Iglehart [64].

6.9 Bibliographical Notes IV

Other Topics

- Motif finding via Gibbs sampling: Lawrence *et al.* [119]; Liu, Neuwald & Lawrence [128].
- Siegmund duality and stable simulation: Siegmund [191]; Lorek & Szekli [130].
- Ising model, phase transitions, and critical phenomena: see textbooks by Grimmett [68] or Georgii [63].

6.10 Selected Exercises I

Exercise 6.T.7 (Riffle Shuffle)

Let (X_k) be a Markov chain on $\mathcal{S} = S_n$ (the set of all permutations of n elements) corresponding to the Riffle Shuffle, and let $(\hat{X}_k)$ be its time-reversal. Show that for all initial distributions μ (mu) and all $k \geq 0$:

$$d_{\text{TV}}(\mu \mathbf{P}_X^k, \pi) = d_{\text{TV}}(\mu \mathbf{P}_{\hat{X}}^k, \pi),$$

where π is the uniform distribution on S_n . This means the riffle shuffle and its time-reversal converge to uniformity at exactly the same rate, in total variation distance.

6.10 Selected Exercises II

Exercise 6.T.8 (Total Variation Distance)

Show that for two probability vectors μ and ν on $\mathcal{S}$:

$$d_{\text{TV}}(\mu, \nu) = \max_{A \subseteq \mathcal{S}} |\mu(A) - \nu(A)|.$$

Hint: Take $A = \{s : \mu_s > \nu_s\}$ (the set of states where μ puts more weight than ν) to show the “ $\geq$ ” direction. For the “ $\leq$ ” direction, use the fact that $|\mu(A) - \nu(A)| \leq \frac{1}{2} \sum_s |\mu_s - \nu_s|$ for any A .

6.10 Selected Exercises III

Exercise 6.T.12 (Los Angeles Weather Chain)

Consider $\mathcal{S} = \{\text{rainy}, \text{sunny}\}$ with $\mathbf{P} = \begin{pmatrix} 1/2 & 1/2 \\ 1/10 & 9/10 \end{pmatrix}$.

- (a) Find the stationary distribution.
- (b) Starting from the “rainy” state, compute $d_{\text{TV}}(\mu\mathbf{P}^n, \pi)$ for $n = 1, 2, 3$.

Solution for (a): The stationary equations are $\pi_r = \pi_r/2 + \pi_s/10$ and $\pi_r + \pi_s = 1$. Solving: $\pi_r/2 = \pi_s/10$, so $\pi_s = 5\pi_r$. With $\pi_r + \pi_s = 1$: $\pi_r + 5\pi_r = 1$, giving $\pi_r = 1/6$, $\pi_s = 5/6$.

Interpretation: even though it is sunny most of the time (9/10 chance of staying sunny), the overall stationary probability of a sunny day is 5/6.

6.10 Selected Exercises IV

Exercise 6.T.20 (Gibbs as Special Case of MH)

Show that the Gibbs sampler (Algorithm 44) is a special case of the Metropolis-Hastings algorithm (Algorithm 43) with acceptance ratio always equal to 1 (every proposed move is accepted).

Hint: For the Gibbs proposal, the proposal probability of moving from ξ to ξ' (where ξ and ξ' differ only at vertex v) is $q_{\xi\xi'} = p_{\xi,v}(\xi'(v))/|V|$. Compute the MH acceptance ratio $\pi_{\xi'} q_{\xi'\xi} / (\pi_{\xi} q_{\xi\xi'})$ and show it equals 1 by using the definition of the full conditional $p_{\xi,v}$.

Exercise 6.T.24 (Forward Coupling is Not Sufficient)

Consider a two-state chain $\mathcal{S} = \{1, 2\}$ where state 1 is absorbing ($p_{11} = 1$) and state 2 transitions to 1 ($p_{21} = 1$). The forward coupling time $\tau_f = \min\{n : X_n = X'_n\}$ (the first time two chains started from different states agree) is finite with probability 1 (since both chains reach state 1 in at most 1 step). However, show that $X_{\tau_f}^0$ (the value of the chain at the forward coupling time) is NOT distributed as π (the stationary distribution, which would be uniform if the chain were ergodic).

This illustrates why CFTP must run backward from the past, not forward from the present: forward coupling does not guarantee the chain has reached stationarity.

6.10 Selected Exercises V

Exercise 6.T.35 (Median Trick for Boosting)

If $p > 1/2$ and $Y_1, \dots, Y_R \stackrel{\text{i.i.d.}}{\sim} \text{Bernoulli}(p)$, show that

$$\mathbb{P}\left(\sum_{i=1}^R Y_i > R/2\right) \geq 1 - \delta \quad \text{when } R \geq \frac{\ln(1/\delta)}{2(p - 1/2)^2}.$$

Hint: Apply Hoeffding's inequality to the sum $\sum_{i=1}^R Y_i$. Hoeffding's inequality states:

$\mathbb{P}(\bar{Y} - \mathbb{E}[\bar{Y}] \leq -t) \leq e^{-2Rt^2}$ for bounded random variables. Set $t = p - 1/2 > 0$ and solve for R . This result underlies the FPRAS boosting argument: if each estimate is correct with probability $p > 1/2$, the majority vote of R independent estimates is correct with probability $\geq 1 - \delta$ for appropriate R .

Chapter 6 Summary I

Chapter 6 developed the complete theory and practice of Markov Chain Monte Carlo (MCMC), from basic Markov chain definitions through state-of-the-art algorithms for exact sampling and approximate counting.

Core Algorithms and Their Applications

Algorithm	Section	Application
Metropolis (Alg. 42)	6.3.1	Symmetric proposal, any π
Metropolis-Hastings (Alg. 43)	6.3.1	Asymmetric proposal, any π
Gibbs Sampler (Alg. 44–48)	6.3.2	Product space, full conditionals
CFTP – general (Alg. 49)	6.4.1	Exact sampling from π
CFTP – monotone (Alg. 50)	6.4.3	Efficient exact sampling (2 chains)
FPRAS (Alg. 54–55)	6.5.1	Approximate counting
Motif Gibbs (Alg. 58)	6.6	Bioinformatics motif finding
Stable Simulation (Alg. 59)	6.8	Exact $\pi(A)$ via Siegmund duality

Chapter 6 Summary II

Key Theoretical Results

- 1 **Detailed Balance Condition (Prop. 6.2.5):** If $w_s p_{ss'} = w_{s'} p_{s's}$ for all s, s' , then $\pi_s \propto w_s$ is stationary. This is the master tool for proving correctness of Metropolis, MH, and Gibbs algorithms.
- 2 **Ergodic Theorem (Prop. 6.7.1):** $\hat{Y}_R \rightarrow I$ almost surely for any ergodic chain and any bounded f . This justifies using MCMC for estimation.
- 3 **Markov CLT (Prop. 6.7.5):** $\sqrt{R}(\hat{Y}_R - I) \xrightarrow{d} \mathcal{N}(0, \varsigma^2)$ with $\varsigma^2 = \rho_0 + 2 \sum_{k \geq 1} \rho_k$. This gives confidence intervals for MCMC estimates.
- 4 **CFTP Correctness (Thm. 6.4.2):** Backward coupling from all initial states — using the same random numbers — yields a sample with distribution *exactly* π , with no burn-in bias.
- 5 **FPRAS (Prop. 6.5.4):** Telescoping ratios $z_i/z_{i-1} \geq 1/2$ combined with rapid mixing gives a polynomial-time randomised counter for intractable combinatorial quantities.
- 6 **Siegmund Duality (Sec. 6.8):** $\pi(\{s_j\}^\downarrow) = \mathbb{P}(Z \text{ absorbed at } s_m \mid Z_0 = s_j)$. This converts a stationary probability estimation problem into an absorption probability problem, enabling exact unbiased estimation via independent dual-chain simulations.

The MCMC Paradigm in One Sentence

Build a Markov chain with stationary distribution π that only requires computing *ratios* $\pi_{s'}/\pi_s$ (so the unknown normalising constant always cancels); run it long enough; average the output — and the ergodic theorem guarantees convergence to the true expectation $I = \mathbb{E}_{\pi}[f]$.